

TECNOLÓGICO DE MONTERREY

Reporte del Proyecto

Compilador para CLite con LLVM IR

Repositorio del Proyecto:

<https://github.com/iiRoy/Mini-Compiler/tree/main>

Estudiante

Rodrigo López Guerra | A01737437

Docente

Luciano García Bañuelos

Fecha de entrega:

24 de Mayo del 2026

I. Introducción

El presente reporte describe el desarrollo de un mini compilador implementado en Python, utilizando PLY para el análisis léxico y sintáctico, y llvmlite para la generación de código intermedio LLVM IR. El proyecto tuvo como objetivo construir progresivamente las etapas principales de un compilador: reconocimiento de tokens, definición de una gramática, construcción de un árbol de sintaxis abstracta, validación semántica básica y generación de instrucciones LLVM para representar el comportamiento del programa fuente.

A lo largo del desarrollo, el compilador fue ampliando sus capacidades. Inicialmente se trabajó con declaraciones simples, expresiones aritméticas y retorno de valores. Posteriormente se agregaron tipos de datos como int, float, bool, char, string y void; estructuras de control como if, else, while, do/while, for y switch; funciones con parámetros, llamadas generales, recursión, arreglos, bloques, declaraciones múltiples y soporte para printf. También se incorporó la lectura de archivos de ejemplo para validar el funcionamiento del compilador con distintos programas fuente.

El proyecto permitió comprender cómo cada parte del compilador se conecta con las demás. El lexer transforma el código fuente en tokens; el parser organiza esos tokens siguiendo una gramática; el AST representa la estructura lógica del programa; y el generador de código traduce esa estructura a LLVM IR. Esta separación hizo posible implementar nuevas características de forma gradual, corregir errores específicos y entender mejor el flujo completo desde el código escrito por el usuario hasta una representación intermedia ejecutable u optimizable.

Además de la implementación técnica, el proyecto sirvió para reforzar conceptos importantes de diseño de lenguajes de programación, manejo de tipos, scopes, recursión, estructuras de control y generación de código. Cada nueva funcionalidad implicó resolver problemas concretos, como conflictos en la gramática, validación de tipos, manejo de variables locales, llamadas entre funciones y representación de arreglos en memoria. Por ello, este reporte documenta las etapas principales del desarrollo, el código clave utilizado en cada una y una reflexión personal sobre lo aprendido durante el proceso.

II. Objetivo del Proyecto

El objetivo del proyecto es diseñar e implementar un mini compilador capaz de procesar un lenguaje de programación similar a C, desde el análisis del código fuente hasta la generación de código intermedio en LLVM IR. Para lograrlo, se busca construir las etapas principales de compilación: análisis léxico, análisis sintáctico, construcción del árbol de sintaxis abstracta, validación semántica básica y generación de instrucciones LLVM.

El proyecto también tiene como objetivo comprender de forma práctica cómo se representa internamente un programa. Esto incluye reconocer variables, tipos de datos, expresiones, funciones, arreglos, estructuras de control y llamadas recursivas, para después traducirlas a una representación más cercana a la máquina. A través de esta implementación, se pretende reforzar la relación entre la teoría de compiladores y su aplicación real mediante código.

Además, el compilador busca soportar programas cada vez más completos, incorporando funcionalidades como declaraciones múltiples, funciones con parámetros, llamadas generales, estructuras condicionales, ciclos, switch/case, manejo de printf, arreglos y bloques de código. De esta manera, el proyecto no solo cumple con una gramática base, sino que también amplía sus capacidades para ejecutar ejemplos más representativos y cercanos a programas reales.

III. Alcance del Compilador

El alcance de este proyecto consistió en desarrollar un mini compilador capaz de reconocer, analizar y traducir programas escritos en un lenguaje similar a C hacia código intermedio LLVM IR. El compilador fue construido de manera incremental, agregando funcionalidades poco a poco para comprobar que cada etapa funcionara correctamente antes de avanzar a la siguiente.

El compilador que desarrollé puede reconocer distintos tipos de datos, como int, float, bool, char, string y void. También permite declarar variables, inicializarlas, realizar asignaciones, trabajar con arreglos de tamaño fijo y evaluar expresiones aritméticas, relacionales y lógicas. Además, implementé conversiones básicas de tipos, como la conversión de int a float cuando es necesario.

En cuanto a estructuras de control, el compilador soporta condicionales if e if/else, ciclos while, do/while y for, además de estructuras switch/case/default con soporte para break. También se incluyeron bloques de código como sentencias, sentencias vacías y manejo de scopes locales para controlar correctamente la vida de las variables declaradas dentro de bloques.

Otro aspecto importante del alcance fue la implementación de funciones. El compilador permite definir funciones además de main, utilizar parámetros, retornar valores, declarar funciones void, realizar llamadas generales a funciones y utilizar recursión. Esto permitió probar programas más completos, como factorial, Fibonacci, torres de Hanoi y otros ejemplos recursivos.

También se agregó soporte para printf, de forma que los programas puedan generar salida en pantalla. Para esto, el compilador crea la declaración correspondiente de printf en LLVM IR y maneja strings globales para los formatos de impresión. Además, se incluyó la capacidad de

ignorar directivas de preprocesador como `#include <stdio.h>`, permitiendo que los archivos de ejemplo se parezcan más a programas escritos en C.

Finalmente, integré una función general de compilación para leer archivos desde una carpeta de ejemplos, parsearlos y generar el LLVM IR correspondiente. Esto permitió validar el funcionamiento del compilador con múltiples programas de prueba. Sin embargo, el alcance actual todavía tiene algunas limitaciones, como el soporte incompleto para variables globales en la generación de código, arreglos como parámetros, inicializadores de arreglos con llaves, verificación de rangos en arreglos y soporte completo para el tipo `double`.

IV. Arquitectura General del Proyecto

4.1 Organización de archivos del proyecto

El proyecto fue organizado separando las responsabilidades principales del compilador en archivos y carpetas específicas. Esta organización me permitió trabajar de forma más ordenada y mantener una separación clara entre la definición del lenguaje, la representación interna del programa y las pruebas realizadas.

El archivo principal del proyecto es ***analysis.py***, donde concentré la definición del lexer, la gramática del parser, la lectura de archivos de ejemplo y el flujo general de compilación. En este archivo se reconocen los tokens del lenguaje, se definen las reglas sintácticas y se llama al proceso que transforma el código fuente en LLVM IR.

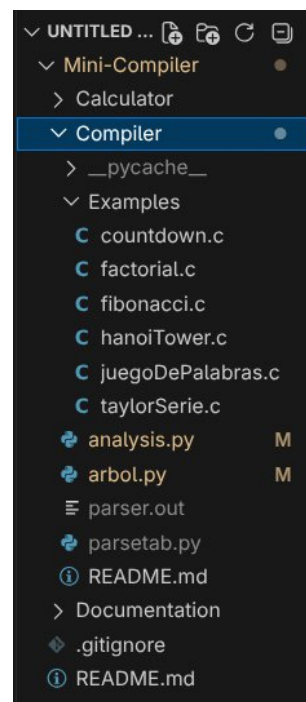
El archivo ***arbol.py*** contiene la definición del árbol de sintaxis abstracta, también conocido como AST. En este archivo implementé las clases que representan los elementos del lenguaje, como programas, funciones, declaraciones, asignaciones, expresiones, llamadas a funciones, ciclos, condicionales, arreglos y estructuras `switch/case`. Además, ahí se encuentra la clase *IRGenerator*, que se encarga de recorrer el AST y generar el código LLVM correspondiente.

También utilicé una carpeta llamada Examples, donde coloqué programas de prueba escritos en el lenguaje soportado por el compilador. Esta carpeta me permitió validar el comportamiento del compilador con distintos ejemplos, como factorial, Fibonacci, torres de Hanoi, juegos de palabras, series de Taylor, uso de `switch`, ciclos, funciones recursivas y arreglos.

Esta organización facilitó el desarrollo incremental del compilador, ya que pude modificar el lexer, el parser, el AST o el generador de código sin mezclar todas las responsabilidades en un solo bloque de código.

4.2 Flujo general de compilación

El flujo general de compilación comienza con la lectura del código fuente. El programa puede escribirse directamente en una cadena de texto o leerse desde un archivo dentro de la carpeta



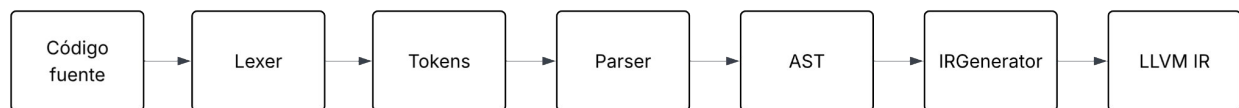
Examples. Para esto implementé una función que carga el contenido del archivo y lo envía al proceso de compilación.

Después de cargar el código fuente, el primer paso es el análisis léxico. En esta etapa, el lexer toma el texto original y lo divide en tokens. Estos tokens representan elementos básicos del lenguaje, como identificadores, palabras reservadas, operadores, literales enteros, literales flotantes, strings, chars, símbolos de agrupación y signos de puntuación.

Una vez que el lexer genera los tokens, el parser los analiza siguiendo la gramática definida. Si la secuencia de tokens cumple con las reglas del lenguaje, el parser construye un árbol de sintaxis abstracta. Este árbol representa la estructura lógica del programa, pero sin depender directamente del texto original.

Después de construir el AST, el compilador realiza la generación de LLVM IR. Para lograrlo, se hacen dos pasadas principales sobre las funciones del programa. En la primera pasada, se crean las firmas LLVM de todas las funciones y se guardan en una tabla de funciones. Esto permite que una función pueda llamar a otra, incluso si la llamada aparece antes de que se genere el cuerpo completo de esa función. También permite manejar llamadas recursivas.

En la segunda pasada, se genera el cuerpo de cada función. Para cada función, se crea un bloque de entrada, un *IRBuilder* y una instancia del *IRGenerator*. El generador recorre las declaraciones, sentencias y expresiones de la función, produciendo instrucciones LLVM para asignaciones, operaciones aritméticas, comparaciones, ciclos, condicionales, llamadas a funciones, arreglos y retornos.



Este flujo me permitió entender cómo un programa escrito en un lenguaje de alto nivel se transforma poco a poco en una representación intermedia más cercana a la ejecución.

4.3 Relación entre lexer, parser, AST e IRGenerator

Las cuatro partes principales del compilador trabajan de forma conectada. Cada una depende del resultado de la etapa anterior y prepara la información necesaria para la siguiente.

El lexer es la primera etapa. Su trabajo consiste en reconocer los elementos mínimos del lenguaje. Por ejemplo, cuando encuentra palabras como **int**, **float**, **if**, **while** o **return**, las clasifica como palabras reservadas. También reconoce identificadores, números, operadores como **+**, **-**, *****, **/**, operadores relacionales como **==**, **!=**, **<=**, **>=**, y operadores lógicos como **&&** y **||**. Además, ignora espacios, comentarios y directivas de preprocesador como **#include <stdio.h>**.

El parser utiliza los tokens generados por el lexer y verifica que estén organizados correctamente según la gramática. Por ejemplo, reconoce cuándo una secuencia de tokens representa una declaración, una función, un ciclo, una condición, una llamada a función o una expresión. Cuando el parser identifica una construcción válida, crea un nodo del AST.

El AST funciona como una representación intermedia del programa. En lugar de trabajar directamente con texto, el compilador trabaja con objetos. Por ejemplo, una operación como:

```
x = a + b;
```

puede representarse mediante nodos de asignación, variable y operación binaria. Esta representación facilita el análisis y la generación de código, porque cada nodo sabe qué información contiene y puede ser visitado por el generador.

El **IRGenerator** es la etapa que recorre el AST y produce LLVM IR. Para hacerlo, utiliza el patrón Visitor: cada clase del AST tiene un método `accept`, y el IRGenerator implementa métodos como **`visit_declaration`**, **`visit_assignment`**, **`visit_binary_op`**, **`visit_if_statement`**, **`visit_while_statement`**, **`visit_call`** y **`visit_return`**. De esta manera, cada tipo de nodo tiene una traducción específica a instrucciones LLVM.

Por ejemplo, una declaración reserva espacio en memoria con `alloca`, una asignación utiliza `store`, una variable se recupera con `load`, una suma genera una instrucción `add` o `fadd`, una comparación genera `icmp` o `fcmp`, y una llamada a función genera una instrucción `call`.

La relación entre estas partes fue fundamental para mantener el proyecto ordenado. El lexer no necesita saber cómo se genera LLVM, el parser no necesita encargarse de la memoria, y el AST no ejecuta directamente el programa. Cada etapa tiene una responsabilidad clara, y el IRGenerator toma la estructura final del AST para producir el código intermedio. Esto hizo posible extender el compilador gradualmente, agregando nuevas funcionalidades sin tener que rehacer todo el proyecto desde cero.

V. Análisis Léxico

El análisis léxico fue la primera etapa que implementé dentro del compilador. Su objetivo fue leer el código fuente como texto y dividirlo en unidades mínimas llamadas tokens. Estos tokens representan elementos importantes del lenguaje, como palabras reservadas, identificadores, literales, operadores y símbolos especiales.

Para esta etapa utilicé la librería PLY, específicamente el módulo `lex`, que permite definir expresiones regulares para reconocer patrones dentro del código fuente. Gracias al lexer, el parser no trabaja directamente con texto plano, sino con una secuencia organizada de tokens que ya tienen un significado dentro del lenguaje.

5.1 Definición de tokens

Los tokens representan los elementos básicos que el compilador puede reconocer. En el proyecto definí tokens para identificadores, números enteros, números flotantes, strings, chars, operadores relacionales y operadores lógicos.

Entre los tokens principales que implementé se encuentran:

```
tokens = ['ID', 'INTLIT', 'FLOATLIT', 'STRINGLIT', 'CHARLIT', 'EQ', 'NE', 'LE', 'GE',  
'AND', 'OR'] + list(reserved.values())
```

El token ID se utiliza para reconocer nombres de variables y funciones. Los tokens INTLIT, FLOATLIT, STRINGLIT y CHARLIT representan literales. También agregué tokens para operadores compuestos, como ==, !=, <=, >=, && y ||.

Además de los tokens, definí una lista de caracteres literales que el lexer reconoce directamente, como operadores aritméticos, paréntesis, llaves, corchetes, comas, punto y coma y otros símbolos necesarios para la gramática:

```
literals = '+-*/%({}[],;=<>!:'
```

Esta separación permitió que operadores simples como +, -, *, /, =, < o > no necesitaran una regla individual como token, mientras que operadores más largos como == o && sí fueron definidos explícitamente.

5.2 Palabras reservadas

Para distinguir entre identificadores normales y palabras propias del lenguaje, implementé un diccionario de palabras reservadas. Esto permitió reconocer instrucciones como tipos de datos, estructuras de control y palabras clave del lenguaje.

Algunas de las palabras reservadas que agregué fueron:

```
reserved = {  
    'int': 'INT',  
    'float': 'FLOAT',  
    'string': 'STRING',  
    'bool': 'BOOL',  
    'char': 'CHAR',  
    'void': 'VOID',  
    'true': 'TRUE',  
    'false': 'FALSE',  
    'return': 'RETURN',  
    'printf': 'PRINTF',  
    'if': 'IF',  
    'else': 'ELSE',  
    'switch': 'SWITCH',  
    'case': 'CASE',  
    'default': 'DEFAULT',  
    'break': 'BREAK',  
    'while': 'WHILE',  
    'do': 'DO',  
    'for': 'FOR'  
}
```

El reconocimiento de identificadores se hizo con la regla:

```
def t_ID(t):
```

```
r'[a-zA-Z_][a-zA-Z_0-9]*'  
lower_value = t.value.lower()  
t.type = reserved.get(lower_value, 'ID')  
  
if t.type == 'TRUE':  
    t.value = True  
elif t.type == 'FALSE':  
    t.value = False  
  
return t
```

Esta regla permite que un identificador empiece con una letra o guion bajo, y que después pueda contener letras, números o guion bajo. Además, convertí el valor a minúsculas para comparar contra el diccionario de palabras reservadas. De esta forma, el lexer puede reconocer palabras como true, false, if, while o return sin confundirlas con nombres de variables.

También convertí directamente los valores booleanos true y false a valores de Python (True y False), para que más adelante el AST y el generador de código pudieran trabajar con ellos de manera más sencilla.

5.3 Reconocimiento de literales

Los literales son valores escritos directamente en el código fuente. En el compilador implementé reconocimiento para enteros, flotantes, strings, chars y booleanos.

Para los enteros utilicé una expresión regular que reconoce uno o más dígitos:

```
def t_INTLIT(t):  
    r'[0-9]+'  
    t.value = int(t.value)  
    return t
```

Con esta regla, valores como 10, 0 o 123 son reconocidos como literales enteros y convertidos a tipo int de Python.

Para los números flotantes utilicé:

```
def t_FLOATLIT(t):  
    r'[0-9]+\.[0-9]+'  
    t.value = float(t.value)  
    return t
```

Esto permitió reconocer valores como 2.5, 0.5 o 10.0, convirtiéndolos a float.

Los strings fueron reconocidos entre comillas dobles:

```
def t_STRINGLIT(t):  
    r'"([^"\\]|\\.)*"'  
    t.value = bytes(t.value[1:-1], "utf-8").decode("unicode_escape")  
    return t
```

Esta regla elimina las comillas externas y permite interpretar secuencias de escape como `\n`. Esto fue importante para que `printf` pudiera recibir formatos de impresión como:

```
printf("Resultado = %d\n", x);
```

Para los caracteres individuales implementé CHARLIT:

```
def t_CHARLIT(t):
    r"'([^\\"|\\.)'"
    char_text = t.value[1:-1]
    char_text = bytes(char_text, "utf-8").decode("unicode_escape")

    if len(char_text) != 1:
        raise ValueError("Un char debe contener exactamente un carácter")

    t.value = ord(char_text)
    return t
```

En este caso, el carácter se convierte a su valor numérico ASCII usando `ord`. Por ejemplo, 'A' se convierte en 65. Esto facilitó su representación en LLVM IR como un entero de 8 bits (i8).

Finalmente, los literales booleanos se manejan desde la regla de identificadores, ya que `true` y `false` se encuentran dentro del diccionario de palabras reservadas. Cuando el lexer los reconoce, cambia su valor a `True` o `False`.

5.4 Manejo de comentarios

También agregué soporte para ignorar comentarios, ya que estos no deben afectar el análisis del programa. Implementé comentarios de una sola línea y comentarios multilínea.

Los comentarios de una sola línea se reconocen con:

```
def t_COMMENT_SINGLELINE(t):
    r'//.*'
    pass
```

Para comentarios multilínea utilicé:

```
def t_COMMENT_MULTILINE(t):
    r'/\*(.|\n)*?\n*/'
    t.lexer.lineno += t.value.count('\n')
    pass
```

Además, actualicé el número de línea contando los saltos de línea dentro del comentario. Esto fue importante para que los errores sintácticos o léxicos posteriores siguieran mostrando una línea aproximada correcta.

5.5 Manejo de directivas de preprocesador

Como algunos ejemplos estaban escritos con una estructura parecida a C real, también agregué soporte para ignorar directivas de preprocesador como:

```
#include <stdio.h>
```

Estas directivas no forman parte de la gramática del lenguaje que implementé, y en este proyecto no era necesario procesarlas realmente. En particular, aunque los programas usen `#include <stdio.h>`, el compilador ya declara `printf` directamente en LLVM IR, por lo que no necesita leer la biblioteca estándar de C.

Para ignorarlas agregué una regla como:

```
def t_PREPROCESSOR(t):  
    r'\#[^\n]*'  
    pass
```

Con esto, cualquier línea que empiece con `#` se descarta durante el análisis léxico. Esto permitió que los archivos de ejemplo se parecieran más a programas escritos en C, sin que el parser fallara por encontrar símbolos como `#`, `<` o `>` dentro de una directiva.

5.6 Manejo de errores léxicos

Para manejar caracteres no reconocidos, implementé la función `t_error`. Esta función se ejecuta cuando el lexer encuentra un carácter que no coincide con ninguna regla definida.

La regla utilizada fue:

```
def t_error(t):  
    print(f"Illegal character '{t.value[0]}'")  
    t.lexer.skip(1)
```

Con esto, si el lexer encuentra un símbolo inválido, muestra un mensaje indicando cuál fue el carácter problemático y después avanza para continuar el análisis. Esto fue útil durante el desarrollo porque permitió detectar rápidamente caracteres que no estaban contemplados en la gramática.

El manejo de errores léxicos fue importante porque me ayudó a identificar problemas en los programas de prueba, como símbolos no soportados, palabras escritas incorrectamente o caracteres especiales que todavía no estaban definidos en el lexer. A partir de estos errores fui ampliando el lenguaje, agregando nuevos tokens y ajustando reglas para que el compilador pudiera reconocer programas más completos.

VI. Análisis Sintáctico

El análisis sintáctico fue la segunda etapa principal del compilador. Después de que el lexer transforma el código fuente en tokens, el parser se encarga de verificar que esos tokens estén organizados correctamente según la gramática del lenguaje. Para esta parte utilicé PLY con el módulo `yacc`, definiendo reglas gramaticales mediante funciones de Python.

El objetivo del parser no fue únicamente validar que el programa estuviera bien escrito, sino también construir el árbol de sintaxis abstracta. Cada vez que el parser reconoce una estructura válida, como una declaración, una función, una asignación o una expresión, crea un nodo del AST que después será utilizado por el generador de LLVM IR.

6.1 Estructura general de la gramática

La gramática del compilador se organizó de forma incremental, comenzando por la estructura más general del programa y después bajando hacia elementos más específicos como declaraciones, sentencias, expresiones, factores y literales.

La regla inicial del parser es Program, que representa el programa completo:

```
def p_Program(p):
    """
    Program : ProgramItems
    """
    p[0] = Program(p[1])
```

A partir de esta regla, el programa se compone de una lista de elementos llamados ProgramItems. Esto me permitió soportar más de una función dentro del mismo archivo y también preparar la gramática para aceptar declaraciones globales.

```
def p_ProgramItems_one(p):
    """
    ProgramItems : ProgramItem
    """
    p[0] = [p[1]]
```

```
def p_ProgramItems_many(p):
    """
    ProgramItems : ProgramItems ProgramItem
    """
    p[1].append(p[2])
    p[0] = p[1]
```

Cada ProgramItem puede representar una función o una declaración global. Para evitar ambigüedades entre una función y una variable global, separé la regla en una parte inicial común y una cola (ProgramItemTail) que decide qué tipo de elemento se está leyendo.

```
def p_ProgramItem_typed(p):
    """
    ProgramItem : Type ID ProgramItemTail
    """
```

Esto permite que el parser lea primero el tipo y el identificador, y después determine si lo siguiente es una función, un arreglo global o una declaración global simple. Esta decisión fue importante porque tanto una función como una variable global pueden iniciar con algo como:

```
int nombre
```

La diferencia aparece después: si viene (es una función, si viene ;, , o [es una declaración.

6.2 Declaración de funciones

Para la declaración de funciones, implementé reglas que permiten funciones con tipo de retorno, nombre, parámetros y un bloque de código. La gramática soporta funciones que retornan un tipo normal, como int, float, bool o char, y también funciones void.

Para funciones con tipo normal, se utiliza:

```
def p_ProgramItem_typed(p):  
    """  
    ProgramItem : Type ID ProgramItemTail  
    """
```

Y la parte que identifica si se trata de una función es:

```
def p_ProgramItemTail_function(p):  
    """  
    ProgramItemTail : '(' Parameters ')' BlockStatement  
    """  
    p[0] = ("function", p[2], p[4])
```

Para funciones void, agregué una regla separada:

```
def p_ProgramItem_void_function(p):  
    """  
    ProgramItem : VOID ID '(' Parameters ')' BlockStatement  
    """  
    p[0] = FunctionDefinition(  
        p[1],  
        p[2],  
        p[4],  
        p[6].decls,  
        p[6].stmts  
    )
```

También agregué soporte para parámetros. Los parámetros pueden ser una lista de elementos o estar vacíos. Además, permití void como forma de indicar que una función no recibe parámetros:

```
def p_Parameters(p):  
    """  
    Parameters : ParameterList
```

```
        | VOID
        | empty
    """
    if p[1] is None:
        p[0] = []
    elif p[1] == "void":
        p[0] = []
    else:
        p[0] = p[1]
```

Cada parámetro se reconoce con:

```
def p_Parameter(p):
    """
    Parameter : Type ID
    """
    p[0] = Parameter(p[1], p[2])
```

6.3 Declaraciones de variables

Las declaraciones de variables se manejan dentro de la regla Declarations. Esta regla permite reconocer cero o más declaraciones al inicio de un bloque.

```
def p_Declarations(p):
    """
    Declarations : Declarations Declaration
                  | empty
    """
    if len(p) == 2:
        p[0] = Declarations([])
    else:
        p[1].declarations.extend(p[2])
        p[0] = p[1]
```

También agregué soporte para inicialización directa. La regla principal fue:

```
def p_Declaration(p):
    """
    Declaration : Type DeclaratorList ';'
                | Type ID '=' Expression ';'
    """
    if len(p) == 4:
        p[0] = p[2]
        for decl in p[0]:
```

```
        decl.var_type = p[1]
    else:
        p[0] = [Declaration(p[1], p[2], p[4])]
```

Con esto pude manejar tanto declaraciones simples como declaraciones con inicializador. Cuando una variable tiene inicializador, el parser guarda la expresión correspondiente dentro del nodo Declaration, para que después el generador de código pueda evaluarla y almacenarla en memoria.

6.4 Declaraciones múltiples

Una mejora importante fue agregar soporte para declaraciones múltiples en una sola línea. Esto permite escribir código como:

```
int a, b, c;
```

Para lograrlo, implementé una lista de declaradores:

```
def p_DeclaratorList_one(p):
    """
    DeclaratorList : Declarator
    """
    p[0] = [p[1]]
```

```
def p_DeclaratorList_many(p):
    """
    DeclaratorList : DeclaratorList ',' Declarator
    """
    p[1].append(p[3])
    p[0] = p[1]
```

Cada declarador puede ser una variable normal o un arreglo:

```
def p_Declarator_variable(p):
    """
    Declarator : ID
    """
    p[0] = Declaration(None, p[1])
```

La razón por la que inicialmente el tipo se guarda como None es porque el tipo está al inicio de toda la declaración. Por ejemplo:

```
int a, b, c;
```

El tipo int aplica a todos los identificadores. Por eso, después de construir la lista, asigno el tipo a cada declaración:

```
for decl in p[0]:
    decl.var_type = p[1]
```

Esto hizo que el compilador pudiera aceptar código más cercano a C y evitar repetir el tipo en múltiples líneas.

6.5 Declaración y acceso a arreglos

También implementé arreglos de tamaño fijo. En la gramática, un arreglo se declara con un identificador seguido de corchetes y un entero:

```
int arr[5];
```

La regla para declaradores de arreglo es:

```
def p_Declarator_array(p):
    """
    Declarator : ID '[' INTLIT '['
    """
    p[0] = Declaration(None, p[1], array_size=p[3])
```

El tamaño del arreglo se guarda dentro del nodo Declaration mediante el atributo array_size. Esto permite que, durante la generación de código, el compilador sepa si una declaración corresponde a una variable normal o a un arreglo.

Para acceder a un elemento del arreglo, agregué la regla ArrayAccess:

```
def p_ArrayAccess(p):
    """
    ArrayAccess : ID '[' Expression '['
    """
    p[0] = ArrayAccess(p[1], p[3])
```

El acceso a arreglos se integró tanto en asignaciones como en expresiones. En una asignación, el lado izquierdo puede ser una variable normal o un acceso a arreglo:

```
def p_Assignment(p):
    """
    Assignment : ID '=' Expression ';'
                | ArrayAccess '=' Expression ';'
    """
    p[0] = Assignment(p[1], p[3])
```

Y dentro de expresiones, el arreglo se maneja como un Primary:

```
def p_Primary_array_access(p):
    """
    Primary : ArrayAccess
    """
    p[0] = p[1]
```

Esto permitió que los arreglos funcionaran tanto para lectura como para escritura.

6.6 Sentencias soportadas

La regla Statement concentra las instrucciones que el lenguaje permite dentro de una función o bloque. A lo largo del proyecto fui ampliando esta regla conforme agregaba nuevas funcionalidades.

La regla quedó estructurada así:

```
def p_Statement(p):  
    """  
    Statement : Assignment  
               | Return  
               | Print  
               | IfStatement  
               | SwitchStatement  
               | Break  
               | WhileStatement  
               | DoWhileStatement  
               | ForStatement  
               | CallStatement  
               | BlockStatement  
               | EmptyStatement  
    """  
    p[0] = p[1]
```

Con esto, el compilador puede reconocer asignaciones, retornos, llamadas a printf, condicionales, ciclos, llamadas a funciones, bloques, switch/case, break y sentencias vacías.

Las sentencias se agrupan dentro de Statements, que representa cero o más instrucciones:

```
def p_Statements(p):  
    """  
    Statements : Statements Statement  
               | empty  
    """  
    if len(p) == 2:  
        p[0] = Statements([])  
    else:  
        p[1].statements.append(p[2])  
        p[0] = p[1]
```

Esto permitió que cada bloque o función pudiera contener una lista de instrucciones, y que estas se procesaran en orden durante la generación de LLVM IR.

6.7 Bloques como sentencias

Una funcionalidad que agregué fue permitir que un bloque completo pudiera funcionar como una sentencia. Esto significa que el lenguaje acepta código como:

```
{  
    int x;  
    x = 10;  
}
```

La regla correspondiente fue:

```
def p_BlockStatement(p):  
    """  
    BlockStatement : '{' Declarations Statements '}'  
    """  
    p[0] = BlockStatement(p[2], p[3])
```

Este bloque incluye declaraciones y sentencias. Esto fue importante porque estructuras como if, while, for y do/while pueden ejecutar bloques completos de código.

Además, al implementar bloques como nodos del AST, pude manejar scopes locales en el generador de código. Es decir, las variables declaradas dentro de un bloque no deberían estar disponibles fuera de él.

6.8 Empty statements

También agregué soporte para sentencias vacías. Una sentencia vacía es simplemente un punto y coma. La regla fue:

```
def p_EmptyStatement(p):  
    """  
    EmptyStatement : ';'   
    """  
    p[0] = EmptyStatement()
```

Aunque una sentencia vacía no genera instrucciones LLVM, sí es válida sintácticamente. Esto permite escribir código como:

```
while (x < 10)  
    ;
```

o incluir un ; aislado dentro de una función sin que el parser falle. Esta funcionalidad también ayudó a que el lenguaje fuera más flexible y parecido a C.

6.9 Expresiones y precedencia de operadores

Una de las partes más importantes del parser fue la gramática de expresiones. Inicialmente las condiciones estaban separadas de las expresiones aritméticas, pero después reestructuré la gramática para que Expression fuera la raíz de todas las expresiones, incluyendo operaciones lógicas, comparaciones y operaciones aritméticas.

La jerarquía quedó así:

```
Expression  -> ||
Conjunction -> &&
Equality    -> == !=
Relation    -> < > <= >=
Addition    -> + -
Term        -> * / %
Factor      -> unary
Primary     -> variables, literales, llamadas, arreglos y grupos
```

La regla superior es Expression, que maneja el operador lógico ||:

```
def p_Expression(p):
    """
    Expression : Expression OR Conjunction
                | Conjunction
    """
    if len(p) == 2:
        p[0] = p[1]
    else:
        p[0] = LogicalOp("||", p[1], p[3])
```

Después está Conjunction, que maneja &&:

```
def p_Conjunction(p):
    """
    Conjunction : Conjunction AND Equality
                | Equality
    """
    if len(p) == 2:
        p[0] = p[1]
    else:
        p[0] = LogicalOp("&&", p[1], p[3])
```

Las comparaciones de igualdad se manejan con:

```
def p_Equality(p):
    """
    Equality : Relation EQ Relation
             | Relation NE Relation
             | Relation
    """
    if len(p) == 2:
        p[0] = p[1]
```

```
else:
    p[0] = CompareOp(p[2], p[1], p[3])
```

Las comparaciones relacionales se manejan con:

```
def p_Relation(p):
    """
    Relation : Addition '<' Addition
              | Addition '>' Addition
              | Addition LE Addition
              | Addition GE Addition
              | Addition
    """
    if len(p) == 2:
        p[0] = p[1]
    else:
        p[0] = CompareOp(p[2], p[1], p[3])
```

La suma y resta se manejan en Addition:

```
def p_Addition(p):
    """
    Addition : Addition '+' Term
              | Addition '-' Term
              | Term
    """
    if len(p) == 2:
        p[0] = p[1]
    else:
        p[0] = BinaryOp(p[2], p[1], p[3])
```

La multiplicación, división y módulo se manejan en Term:

```
def p_Term(p):
    """
    Term : Term '*' Factor
          | Term '/' Factor
          | Term '%' Factor
          | Factor
    """
    if len(p) == 2:
        p[0] = p[1]
    else:
        p[0] = BinaryOp(p[2], p[1], p[3])
```

Finalmente, Factor y Primary permiten literales, variables, llamadas, arreglos, expresiones agrupadas y operadores unarios:

```
def p_Factor_unary_minus(p):  
    """  
  
    Factor : '-' Primary  
    """  
  
    p[0] = UnaryOp("-", p[2])
```

```
def p_Factor_not(p):  
    """  
  
    Factor : '!' Primary  
    """  
  
    p[0] = UnaryOp("!", p[2])
```

6.10 Llamadas a funciones

Para soportar llamadas a funciones, agregué una regla Call. Una llamada está formada por un identificador, paréntesis y una lista opcional de argumentos:

```
def p_Call(p):  
    """  
  
    Call : ID '(' OptionalArguments ')'  
    """  
  
    p[0] = Call(p[1], p[3])
```

Los argumentos pueden ser uno o más expresiones:

```
def p_Arguments_one(p):  
    """  
  
    Arguments : Expression  
    """  
  
    p[0] = [p[1]]
```

```
def p_Arguments_many(p):  
    """  
  
    Arguments : Arguments ',' Expression  
    """  
  
    p[1].append(p[3])  
    p[0] = p[1]
```

Y también pueden estar vacíos:

```
def p_optionalArguments(p):  
    """  
    OptionalArguments : Arguments  
                        | empty  
    """  
    if p[1] is None:  
        p[0] = []  
    else:  
        p[0] = p[1]
```

Las llamadas pueden aparecer como expresiones, por ejemplo:

```
x = factorial(5);
```

o como sentencias independientes:

```
printWord(1);
```

Para el segundo caso agregué CallStatement:

```
def p_CallStatement(p):  
    """  
    CallStatement : Call ';' '  
    """  
    p[0] = CallStatement(p[1])
```

Esto fue necesario especialmente para funciones void, ya que una función que no retorna valor no debe usarse dentro de una expresión, pero sí puede llamarse como una sentencia.

6.11 Manejo de errores sintácticos

Para manejar errores sintácticos implementé la función `p_error`. Esta función se ejecuta cuando el parser encuentra una secuencia de tokens que no coincide con ninguna regla de la gramática.

La versión inicial imprimía el error y permitía que el parser intentara continuar. Sin embargo, esto podía generar errores confusos más adelante, porque el AST podía quedar incompleto. Por eso, decidí lanzar una excepción directamente:

```
def p_error(p):  
    if p:  
        raise SyntaxError(f"Syntax error: token={p.type}, value={p.value},  
                           line={p.lineno}")  
    else:  
        raise SyntaxError("Syntax error: unexpected end of input")
```

Con esta implementación, si el parser encuentra un error, el proceso se detiene inmediatamente y muestra el token, el valor y la línea donde ocurrió el problema.

Esto fue útil para detectar errores como funciones mal declaradas, tipos no soportados, sintaxis inválida en declaraciones, llamadas mal formadas o estructuras incompletas.

También tuve que resolver conflictos sintácticos, como el caso clásico del else ambiguo. Para esto agregué precedencia:

```
precedence = (  
    ('nonassoc', 'IFX'),  
    ('nonassoc', 'ELSE'),  
)
```

Y apliqué esa precedencia en la regla del if sin else:

```
def p_IfStatement_if(p):  
    """  
    IfStatement : IF '(' Expression ')' Statement %prec IFX  
    """  
    p[0] = IfStatement(p[3], Statements([p[5]]), None)
```

Esto ayudó a que el parser asocia correctamente el else con el if más cercano, evitando conflictos durante la construcción de las tablas LALR.

En general, el análisis sintáctico fue una de las etapas más importantes del proyecto, porque definió qué programas podía aceptar el compilador y cómo se construía la representación interna que después sería traducida a LLVM IR.

VII. Árbol de Sintaxis Abstracta

Después de definir el análisis léxico y sintáctico, la siguiente etapa fue construir una representación interna del programa. Para esto implementé un árbol de sintaxis abstracta, también conocido como AST, que resume la estructura del código fuente en objetos de Python. Esta representación elimina detalles de sintaxis concreta, como paréntesis o signos de puntuación, y conserva únicamente la información necesaria para analizar y generar código.

El AST fue diseñado en el archivo `arbol.py`. Cada clase representa una parte del lenguaje: programas, funciones, declaraciones, sentencias, expresiones, literales, llamadas, arreglos y estructuras de control. Esta separación hizo que el compilador fuera más fácil de extender, porque al agregar una nueva construcción del lenguaje solo tenía que crear un nodo y definir cómo debía visitarse durante la generación de LLVM IR.

La implementación del AST también me permitió aplicar el patrón Visitor. Cada nodo tiene un método `accept` que recibe un visitor, y el `IRGenerator` implementa los métodos necesarios para recorrer esos nodos. De esta forma, el árbol no genera directamente el código, sino que delega esa responsabilidad a una clase especializada.

7.1 Propósito del AST

El propósito principal del AST fue convertir el resultado del parser en una estructura más fácil de procesar. El parser reconoce reglas gramaticales, pero el generador de código necesita trabajar con objetos que indiquen con claridad qué representa cada parte del programa. Por ejemplo, una asignación, una suma o un `return` son conceptos diferentes y por eso los representé con clases diferentes.

Esta decisión fue importante porque el código fuente original contiene muchos elementos que son útiles para el parser, pero no necesariamente para la generación de LLVM IR. Por ejemplo, los paréntesis, comas, llaves y puntos y coma ayudan a reconocer la sintaxis, pero no son entidades que deban producir instrucciones por sí mismas. El AST simplifica esa información y deja solo la estructura lógica.

Gracias al AST pude separar dos responsabilidades: reconocer la sintaxis y generar código. El parser crea nodos como `BinaryOp`, `IfStatement` o `FunctionDefinition`, mientras que el `IRGenerator` decide cómo traducir esos nodos a instrucciones LLVM. Esta separación hizo que el proyecto fuera más claro y permitió depurar errores de manera más ordenada.

```
class ASTNode(ABC):
    @abstractmethod
    def accept(self, visitor: Visitor) -> None:
        pass
```

7.2 Nodos principales del programa

Los nodos principales del programa son `Program` y `FunctionDefinition`. `Program` funciona como la raíz del árbol y almacena los elementos encontrados a nivel global. En mi implementación separé los elementos del programa en dos listas: `functions` y `globals`. Esto permitió que el compilador identificara con facilidad qué elementos eran funciones y cuáles eran declaraciones globales.

`FunctionDefinition` representa una función completa. Este nodo guarda el tipo de retorno, el nombre de la función, la lista de parámetros, las declaraciones locales y las sentencias que

forman el cuerpo. Esta estructura fue esencial para generar LLVM IR, porque cada función necesita una firma, un bloque de entrada, parámetros y un cuerpo de instrucciones.

La separación entre Program y FunctionDefinition también ayudó a implementar el proceso de dos pasadas. En la primera pasada se recorren las funciones para crear sus firmas en LLVM, y en la segunda pasada se generan los cuerpos. Esto fue necesario para permitir llamadas entre funciones y llamadas recursivas, como en los ejemplos de Fibonacci, factorial y torres de Hanoi.

```
class Program(ASTNode):
    def __init__(self, items):
        self.items = items
        self.functions = []
        self.globals = []

class FunctionDefinition(ASTNode):
    def __init__(self, return_type, name, params, decls, stmts):
        self.return_type = return_type
        self.name = name
        self.params = params
        self.decls = decls
        self.stmts = stmts
```

7.3 Nodos de declaraciones

Las declaraciones fueron representadas principalmente con los nodos Declaration, Declarations y GlobalDeclaration. Declaration almacena el tipo de dato, el nombre de la variable, un posible inicializador y, en caso de arreglos, el tamaño del arreglo. Esta estructura me permitió usar una sola clase para variables normales y arreglos.

Declarations funciona como un contenedor de declaraciones locales. El parser lo utiliza para agrupar las variables declaradas al inicio de una función o de un bloque. Como el lenguaje permite múltiples declaraciones, este nodo evita manejar declaraciones sueltas y facilita que el IRGenerator las procese en orden.

GlobalDeclaration se agregó para que el parser pudiera aceptar declaraciones globales. Aunque la generación completa de LLVM para variables globales quedó como una limitación actual, el AST ya puede representarlás. Esta decisión dejó preparado el proyecto para una mejora futura sin tener que rediseñar toda la gramática.

7.4 Nodos de statements

Los statements representan instrucciones que se ejecutan dentro de una función o bloque. En el AST implementé nodos como Assignment, Print, IfStatement, WhileStatement, DoWhileStatement, ForStatement, SwitchStatement, Break, Return, CallStatement, BlockStatement y EmptyStatement. Cada uno encapsula la información específica de su construcción.

Por ejemplo, IfStatement guarda una condición, un bloque de sentencias para la rama verdadera y opcionalmente un bloque para la rama else. WhileStatement guarda una condición y un cuerpo. SwitchStatement guarda la expresión evaluada, una lista de casos y un bloque default opcional. Esta organización hace que cada estructura de control tenga una representación clara.

El nodo Statements funciona como contenedor de instrucciones. Esta clase fue necesaria porque una función, un bloque, una rama de if o un caso de switch pueden contener varias sentencias. Al recorrer este contenedor, el IRGenerator puede generar código en el mismo orden en el que aparece en el programa fuente.

7.5 Nodos de expresiones

Las expresiones se representan mediante nodos como BinaryOp, CompareOp, LogicalOp, UnaryOp, Variable, Literal y ArrayAccess. Estos nodos modelan las operaciones que producen valores. Por ejemplo, una suma se representa con BinaryOp, una comparación con CompareOp y una operación lógica como && o || con LogicalOp.

BinaryOp almacena el operador y sus dos operandos. Esto permitió representar operaciones aritméticas como suma, resta, multiplicación, división y módulo. CompareOp funciona de forma similar, pero se utiliza para operadores relacionales y de igualdad, como <, <=, >, >=, == y !=.

UnaryOp fue importante para implementar operadores como el signo negativo y la negación lógica. Literal guarda valores constantes como enteros, flotantes, chars, strings y booleanos. Variable representa el uso de una variable declarada, mientras que ArrayAccess representa la lectura de una posición específica de un arreglo.

7.6 Nodos para funciones y llamadas

Para soportar funciones implementé los nodos FunctionDefinition, Parameter, Call y CallStatement. FunctionDefinition representa la declaración completa de una función. Parameter almacena el tipo y el nombre de cada parámetro, lo cual después se usa para construir la firma LLVM de la función.

Call representa una llamada a función dentro de una expresión. Este nodo guarda el nombre de la función y la lista de argumentos. Por ejemplo, una expresión como factorial(5) se convierte en un nodo Call con el nombre factorial y un argumento literal entero.

CallStatement se agregó para distinguir llamadas usadas como sentencias independientes. Esto fue especialmente necesario para funciones void, porque una función que no retorna valor puede llamarse como instrucción, pero no debe usarse como parte de una expresión o asignación.

7.7 Nodos para arreglos

Los arreglos se representaron usando dos elementos principales: Declaration con el atributo array_size y el nodo ArrayAccess. Cuando una declaración tiene array_size distinto de None, el compilador la interpreta como un arreglo de tamaño fijo. Esto evita crear una clase separada solo para la declaración de arreglos.

ArrayAccess guarda el nombre del arreglo y la expresión usada como índice. Esta expresión puede ser un literal, una variable o una operación que produzca un entero. Por ejemplo, arr[i + 1] puede representarse porque el índice se guarda como un nodo de expresión completo.

Esta representación fue suficiente para implementar lectura y escritura de arreglos locales. El IRGenerator puede usar el nombre del arreglo para buscar su dirección en la tabla de símbolos, evaluar el índice y generar un getelementptr para apuntar al elemento correspondiente.

```
class ArrayAccess(ASTNode):
    def __init__(self, name: str, index: ASTNode) -> None:
        self.name = name
        self.index = index
```

VIII. Generación de Código LLVM IR

La generación de código intermedio fue la etapa en la que el AST se transformó en instrucciones LLVM IR usando llvmlite. Esta etapa fue la más cercana al comportamiento real del programa,

porque aquí se definió cómo se reservan variables, cómo se evalúan expresiones, cómo se construyen saltos condicionales y cómo se generan llamadas a funciones.

El documento de apoyo sobre LLVM IR para estructuras de control fue útil para entender el uso de bloques básicos, instrucciones `alloca`, `load`, `store`, `branch` y `cbranch`. En particular, la generación de ciclos se basa en crear bloques para la condición, el cuerpo y la salida, y después conectar esos bloques con saltos condicionales e incondicionales.

En mi compilador, la clase encargada de esta etapa fue `IRGenerator`. Esta clase implementa los métodos del `Visitor` y mantiene estructuras internas como una pila temporal, una tabla de símbolos, una pila de scopes, una tabla de funciones y una lista de bloques de salida para `break`. Con estos elementos, el generador puede traducir cada nodo del AST a LLVM IR.

8.1 Configuración inicial de tipos LLVM

Antes de generar cualquier función, configuré los tipos básicos de LLVM que usa el compilador. El tipo `int` se representó con `ir.IntType(32)`, `float` con `ir.FloatType()`, `bool` con `ir.IntType(1)`, `char` con `ir.IntType(8)` y `string` como un apuntador a `i8`. Esta configuración permitió mapear los tipos del lenguaje a tipos válidos de LLVM.

Esta configuración se realiza dentro de la función general de compilación y también se entrega al `IRGenerator`. De esta manera, cada visita del AST puede usar los mismos tipos y no necesita reconstruirlos. Esto evita inconsistencias, por ejemplo, que una variable `int` se declare con un tipo y luego se compare o almacene usando otro.

También implementé una función `get_llvm_type` para convertir nombres de tipo del lenguaje a tipos LLVM. Esta función se usa especialmente durante la primera pasada, cuando se crean las firmas de todas las funciones. Si el tipo no existe, se lanza un error para evitar generar código inválido.

```
intType = ir.IntType(32)
floatType = ir.FloatType()
stringType = ir.IntType(8).as_pointer()
boolType = ir.IntType(1)
charType = ir.IntType(8)
```

8.2 Tabla de símbolos

La tabla de símbolos fue una de las estructuras internas más importantes del `IRGenerator`. La utilicé para guardar la dirección de memoria de cada variable declarada, su tipo y, en caso de arreglos, su tamaño. Esta información se guarda en un diccionario llamado `symbol_table`.

Cada entrada de la tabla tiene la forma `(ptr, tipo, array_size)`. Para variables normales, `array_size` es `None`. Para arreglos, `array_size` contiene el tamaño declarado. Esta decisión fue importante porque varias operaciones necesitan saber si un identificador representa una variable simple o un arreglo.

La tabla de símbolos se consulta cuando se lee una variable, cuando se realiza una asignación o cuando se accede a un arreglo. Si un nombre no existe en la tabla, el compilador reporta que la variable no ha sido declarada. Esto agrega una validación semántica básica antes de generar LLVM IR.

8.3 Manejo de scopes

Para manejar bloques de código, implementé una pila de scopes llamada `scope_stack`. Cada vez que el compilador entra a un bloque, se agrega un nuevo conjunto a la pila. Las variables declaradas dentro de ese bloque se registran en el scope actual.

Cuando el bloque termina, el compilador elimina de la tabla de símbolos las variables que pertenecían a ese scope. Esto permite que una variable declarada dentro de un bloque no pueda usarse fuera de él. Aunque el manejo de scopes todavía es simple y no soporta shadowing completo, sí evita que variables locales queden vivas fuera de su región.

Esta parte fue importante cuando agregué `BlockStatement` como una sentencia válida. Sin `scope_stack`, las variables declaradas dentro de un bloque de `if`, `while` o un bloque independiente seguirían apareciendo en la tabla de símbolos, lo cual no refleja correctamente el comportamiento esperado de un lenguaje tipo C.

```
def visit_block_statement(self, node: BlockStatement) -> None:
    self.scope_stack.append(set())
    node.decls.accept(self)
    node.stmts.accept(self)
    local_names = self.scope_stack.pop()
    for name in local_names:
        if name in self.symbol_table:
            del self.symbol_table[name]
```

8.4 Generación de funciones

La generación de funciones se hizo en dos pasadas. En la primera pasada, se crean las firmas LLVM de todas las funciones y se guardan en `function_table`. Esta tabla almacena el objeto LLVM de la función, su tipo de retorno y la lista de tipos de sus parámetros.

La primera pasada fue necesaria para soportar llamadas entre funciones y recursión. Por ejemplo, una función como `fib` necesita llamarse a sí misma. Si el cuerpo se generara antes de registrar la firma, el compilador no encontraría la función en `function_table` y marcaría un error.

En la segunda pasada se genera el cuerpo de cada función. Para cada una se crea un bloque entry, un `IRBuilder` y una instancia de `IRGenerator`. Después se visita el nodo `FunctionDefinition`, que genera parámetros, declaraciones y sentencias en orden.

```
function_table = {}
for function_node in tree.functions:
    returnType = get_llvm_type(function_node.return_type)
    param_types = [get_llvm_type(param.var_type) for param in function_node.params]
    fnty = ir.FunctionType(returnType, param_types)
    llvm_func = ir.Function(module, fnty, name=function_node.name)
```

8.5 Generación de parámetros

Los parámetros de una función llegan a LLVM como argumentos de la función. Sin embargo, para tratarlos igual que variables locales, copié cada argumento a memoria usando `alloca` y `store`. De esta forma, el resto del compilador puede cargar los parámetros con `load`, igual que cualquier variable local.

Durante esta etapa también asigné el nombre del parámetro al argumento LLVM. Esto hace que el IR generado sea más legible, porque en lugar de usar nombres anónimos, el código intermedio conserva nombres relacionados con el programa fuente.

Después de reservar memoria para cada parámetro, lo guardé en `symbol_table` con `array_size` en `None`. Esto permitió que los parámetros fueran encontrados por `visit_variable`, por expresiones, por llamadas recursivas y por estructuras de control dentro del cuerpo de la función.

```
def visit_parameters(self, params):
    for param, llvm_arg in zip(params, self.builder.function.args):
        llvm_arg.name = param.name
        ptr = self.builder.alloca(self.get_llvm_type(param.var_type), name=param.name)
        self.builder.store(llvm_arg, ptr)
        self.symbol_table[param.name] = (ptr, param.var_type, None)
```

8.6 Generación de declaraciones

Para generar una declaración de variable normal, el compilador primero obtiene el tipo LLVM correspondiente y luego reserva espacio en memoria con `builder.alloca`. Después registra esa dirección en `symbol_table` para que la variable pueda usarse en expresiones o asignaciones posteriores.

Si la declaración incluye un inicializador, el `IRGenerator` visita la expresión, obtiene el resultado desde la pila temporal y valida que el tipo sea compatible con el tipo de la variable. En caso de una asignación de `int` a `float`, se realiza una conversión implícita usando `sitofp`.

Cuando la variable no tiene inicializador, decidí asignarle un valor por defecto. Para `int` uso 0, para `float` uso 0.0, para `bool` uso `false`, para `char` uso 0 y para `string` uso un apuntador nulo. Esto evita lecturas de memoria sin inicializar dentro de los programas generados.

8.7 Generación de asignaciones

La generación de asignaciones comienza evaluando la expresión del lado derecho. El resultado de esa expresión se deja en la pila temporal como una tupla con valor LLVM y tipo semántico. Después el compilador localiza la variable o el elemento de arreglo al que se va a asignar.

Si el lado izquierdo es una variable normal, se busca en `symbol_table`. Si es un `ArrayAccess`, se llama a `get_array_element_ptr` para obtener la dirección exacta del elemento. Esta distinción fue necesaria porque una asignación a `arr[0]` no se guarda en la dirección base del arreglo, sino en la dirección del elemento.

Antes de emitir el `store`, se valida la compatibilidad de tipos con `cast_assignment_value`. Esto centralizó la lógica de conversión y evitó repetir el mismo bloque de validación en declaraciones, asignaciones normales y asignaciones a arreglos.

```
def visit_assignment(self, node: Assignment) -> None:
    node.expression.accept(self)
    value, value_type = self.stack.pop()
    if isinstance(node.variable, ArrayAccess):
        ptr, target_type = self.get_array_element_ptr(node.variable)
    else:
        ptr, target_type, array_size = self.symbol_table[node.variable]
    value, value_type = self.cast_assignment_value(value, value_type, target_type)
    self.builder.store(value, ptr)
```

8.8 Generación de expresiones aritméticas

Las expresiones aritméticas se generan visitando primero el lado izquierdo y luego el lado derecho del `BinaryOp`. Cada visita deja su resultado en la pila temporal. Después se extraen ambos valores y se decide qué instrucción LLVM debe emitirse según el operador y los tipos involucrados.

Para operaciones con enteros, utilicé instrucciones como `add`, `sub`, `mul`, `sdiv` y `srem`. Para operaciones con flotantes utilicé `fadd`, `fsub`, `fmul` y `fdiv`. Esta separación fue necesaria porque LLVM diferencia claramente entre operaciones enteras y operaciones de punto flotante.

También implementé promoción de int a float cuando uno de los operandos es float. En ese caso, el operando entero se convierte con `sitofp` antes de emitir la operación flotante. Esto permitió expresiones como `a + 2.5` cuando `a` es int.

8.9 Generación de comparaciones

Las comparaciones producen valores booleanos. Para enteros, chars y bools utilicé `icmp_signed`, mientras que para flotantes utilicé `fcmp_ordered`. Esta distinción es importante porque LLVM maneja comparaciones enteras y flotantes con instrucciones diferentes.

Antes de generar la comparación, el compilador valida tipos. No permití comparar strings directamente. Para char y bool exigí que ambos operandos fueran del mismo tipo. Para float permití comparación entre int y float, convirtiendo el int a float cuando era necesario.

El resultado de una comparación se guarda en la pila como tipo bool. Este resultado después puede usarse en if, while, for, do/while o como parte de una expresión lógica. Así, expresiones como `value >= 0 && value != 3` funcionan como combinación de `CompareOp` y `LogicalOp`.

8.10 Generación de operadores lógicos

Los operadores lógicos `&&` y `||` se implementaron con el nodo `LogicalOp`. Para generar código, primero se evalúan ambos operandos y luego se verifica que ambos sean de tipo bool. Si alguno no lo es, el compilador lanza un error de tipo.

Para `&&` utilicé `builder.and_` y para `||` utilicé `builder.or_`. Estas instrucciones operan sobre valores i1 en LLVM, que corresponden al tipo bool del compilador. Esto permitió construir condiciones compuestas para if, while y returns booleanos.

La implementación actual evalúa ambos lados de la expresión lógica antes de aplicar el operador. Esto funciona correctamente para los ejemplos implementados, aunque no implementa short-circuit evaluation. Una mejora futura sería generar bloques adicionales para evaluar el segundo operando solo cuando sea necesario.

8.11 Generación de operadores unarios

Los operadores unarios implementados fueron `!` y `-`. El operador `!` solo se permite sobre valores booleanos. Para generarlo, utilicé `builder.not_`, lo que produce el valor lógico opuesto en LLVM.

El operador `-` se implementó tanto para int como para float. Para enteros, generé una resta desde cero usando `sub`. Para flotantes, generé una resta flotante desde 0.0 usando `fsub`. Esto permitió expresiones como `-1`, `-x` y `-2.5`.

La validación de tipos fue importante en esta etapa. No permití aplicar `-` a strings, chars o bools, ni aplicar `!` a enteros o flotantes. Con esto evité producir IR inválido o semánticamente incorrecto.

```
elif node.op == '-':
    if value_type == 'int':
        zero = ir.Constant(self.intType, 0)
        result = self.builder.sub(zero, value)
    elif value_type == 'float':
        zero = ir.Constant(self.floatType, 0.0)
        result = self.builder.fsub(zero, value)
```

8.12 Generación de arreglos

Los arreglos locales se generan usando `ir.ArrayType` y `alloca`. Cuando una declaración tiene `array_size`, el compilador crea un tipo LLVM como `[5 x i32]` para un arreglo de cinco enteros.

Después guarda la dirección base del arreglo en `symbol_table` junto con su tipo de elemento y tamaño.

También inicialicé cada elemento del arreglo con un valor por defecto. Para lograrlo, recorrí los índices desde 0 hasta `array_size - 1` y generé un `getelementptr` para cada posición. Después emití `store` con el valor inicial correspondiente al tipo del arreglo.

Para leer o escribir un elemento, el compilador evalúa el índice y verifica que sea `int`. Después genera un `getelementptr` con dos índices: el primero para entrar al arreglo y el segundo para seleccionar el elemento. Esta instrucción es la base para manipular arreglos en LLVM IR.

```
elem_ptr = self.builder.gep(
    array_ptr,
    [ir.Constant(self.intType, 0), index_value],
    inbounds=True
)
```

8.13 Generación de llamadas a funciones

Las llamadas a funciones se centralizaron en `emit_call`. Esta función revisa que la función exista en `function_table`, valida la cantidad de argumentos y compara los tipos esperados con los tipos obtenidos al evaluar cada argumento.

Cada argumento se evalúa como una expresión normal. Después se aplica `cast_value` para permitir conversiones válidas, como `int` a `float`, cuando el parámetro esperado es `float`. Finalmente, los valores LLVM se pasan a `builder.call` para generar la llamada.

La diferencia entre `visit_call` y `visit_call_statement` fue importante. `visit_call` se usa cuando la llamada produce un valor y puede aparecer dentro de una expresión. `visit_call_statement` se usa cuando la llamada es una instrucción independiente, lo cual permite usar funciones `void` sin intentar guardar un valor inexistente.

```
def emit_call(self, node: Call):
    llvm_func, return_type, param_types = self.functions[node.name]
    call_args = []
    for arg_node, expected_type in zip(node.args, param_types):
        arg_node.accept(self)
        value, value_type = self.stack.pop()
        value, value_type = self.cast_value(value, value_type, expected_type)
        call_args.append(value)
    return self.builder.call(llvm_func, call_args), return_type
```

8.14 Generación de `printf`

El soporte de `printf` se agregó como una construcción especial del lenguaje. Aunque los archivos fuente pueden incluir `#include <stdio.h>`, el compilador ignora esa directiva y declara `printf` directamente en el módulo LLVM como una función variádica.

La regla `Print` exige que el primer argumento sea un `string`, porque ese `string` funciona como formato de impresión. Los demás argumentos se evalúan como expresiones. Si un argumento es `bool` o `char`, lo extiendo a `int` antes de pasarlo a `printf`, porque las funciones variádicas de C esperan estos valores promovidos.

El uso de `printf` permitió probar programas con salida visible, como Fibonacci, torres de Hanoi y el juego de palabras. Además, obligó a implementar correctamente `strings` globales y conversiones pequeñas para `char` y `bool`.

```
printf_type = ir.FunctionType(self.intType, [self.stringType], var_arg=True)
self.printf = ir.Function(self.module, printf_type, name='printf')
```

8.15 Manejo de strings globales

Los strings se manejan como variables globales constantes dentro del módulo LLVM. Cuando el compilador encuentra un literal string, `create_global_string` crea un arreglo de i8 terminado en cero y lo registra con linkage internal.

Un problema que resolví fue la duplicación de nombres como `.str0`. Esto sucedía porque se creaba un `IRGenerator` nuevo para cada función, y cada uno iniciaba su contador de strings en cero. Para corregirlo, hice que `create_global_string` buscara un nombre libre dentro de `module.globals` antes de crear la variable global.

También calculé el tamaño del string usando los bytes codificados en UTF-8, no solamente `len(text)`. Esto es más correcto porque algunos caracteres pueden ocupar más de un byte. Aunque los ejemplos principales usan strings simples, esta decisión hace la implementación más robusta.

```
while True:
    name = f'.str{self.string_count}'
    self.string_count = self.string_count + 1
    if name not in self.module.globals:
        break
```

IX. Implementación de Tipos de Datos

El compilador soporta varios tipos de datos que se utilizan tanto en el análisis semántico como en la generación de LLVM IR. Cada tipo tiene una representación dentro del lenguaje fuente, una representación interna en el AST y una representación concreta en LLVM.

Los tipos implementados fueron `int`, `float`, `bool`, `char`, `string` y `void`. Además, se agregó conversión implícita de `int` a `float` para operaciones, asignaciones, retornos y llamadas a funciones cuando el contexto lo permite.

El manejo de tipos fue una parte central del proyecto porque evita generar código incorrecto. Cada operación valida qué tipos recibe y decide si debe aceptar la operación, aplicar una conversión o lanzar un error.

9.1 Tipo `int`

El tipo `int` fue el tipo base más utilizado en el compilador. Lo representé en LLVM como `i32` mediante `ir.IntType(32)`. Este tipo se usa para variables enteras, contadores, índices de arreglos, retornos de funciones y operaciones aritméticas enteras.

Las operaciones sobre `int` se generan con instrucciones enteras de LLVM. Para suma, resta y multiplicación utilicé `add`, `sub` y `mul`. Para división utilicé `sdiv`, y para módulo utilicé `srem`. Estas instrucciones producen nuevamente valores enteros.

También utilicé `int` como tipo obligatorio para los índices de arreglos. Antes de generar `getelementptr`, el compilador verifica que el índice sea `int`. Esto simplifica el modelo de memoria y evita accesos con tipos no válidos como `float` o `bool`.

9.2 Tipo `float`

El tipo `float` fue implementado usando `ir.FloatType`. Este tipo permitió manejar números con punto decimal y generar operaciones aritméticas flotantes. Aunque los ejemplos originales de C pueden usar `double`, el compilador trabaja actualmente con `float`.

Cuando una operación involucra un float y un int, el int se convierte a float usando `sitofp`. Esta conversión se aplica en operaciones aritméticas, comparaciones, asignaciones y llamadas a funciones. Con esto, expresiones como `a + 2.5` se pueden generar correctamente.

Las operaciones flotantes usan instrucciones específicas de LLVM: `fadd`, `fsub`, `fmul` y `fdiv`. Esta separación fue importante porque no se puede usar `add` o `sdiv` sobre valores flotantes. LLVM exige que las instrucciones coincidan con los tipos de los operandos.

9.3 Tipo bool

El tipo `bool` se representó como `i1` en LLVM, ya que solo necesita almacenar verdadero o falso. Los literales `true` y `false` se reconocen en el lexer y se transforman a valores booleanos internos antes de llegar al AST.

Los valores `bool` se producen principalmente mediante comparaciones y operadores lógicos. Las estructuras de control como `if`, `while`, `do/while` y `for` exigen que su condición sea `bool`. Si la condición produce otro tipo, el compilador lanza un error.

Para `printf`, los valores `bool` se extienden a `int` usando `zext` antes de pasarse como argumento. Esto permite imprimirlos como valores numéricos cuando se usa un formato entero. Esta conversión solo se hace para impresión, no cambia el tipo semántico del lenguaje.

9.4 Tipo char

El tipo `char` se representó como `i8` en LLVM. En el lexer, los caracteres se leen entre comillas simples y se convierten a su valor ASCII con `ord`. Por ejemplo, `A` se convierte en 65.

Los `chars` pueden declararse, asignarse, pasarse como parámetros y compararse con otros `chars`. Para evitar ambigüedades, el compilador no permite comparar `char` con `int` directamente. Si una comparación involucra `char`, ambos lados deben ser `char`.

Cuando un `char` se pasa a `printf`, se extiende a `i32` con `zext`. Esto fue necesario para usar formatos como `%c`, ya que `printf` es variádico y espera que los argumentos pequeños se promocionen antes de la llamada.

9.5 Tipo string

El tipo `string` se implementó como un apuntador a `i8`. En el lenguaje se usa principalmente para los formatos de `printf`. Los literales `string` se reconocen entre comillas dobles y se transforman en arreglos globales constantes terminados en cero.

No implementé operaciones aritméticas ni comparaciones entre strings. Si un `string` aparece en una comparación o en una operación binaria, el compilador lanza un error. Esta decisión mantiene el lenguaje más simple y evita semánticas complejas de comparación de cadenas.

El uso principal de `string` es servir como primer argumento de `printf`. La función `visit_print` valida que ese primer argumento sea `string` antes de generar la llamada. Esto evita errores como intentar usar un entero como formato de impresión.

9.6 Tipo void

El tipo `void` se usa para funciones que no regresan valor. En LLVM se representa mediante `ir.VoidType`. Este tipo no se permite como tipo de variable, únicamente como tipo de retorno de función.

Para funciones void, el parser permite return sin expresión. En el IRGenerator, si una función void ejecuta return;, se genera ret void. Si termina el cuerpo de la función sin un return explícito, también se puede generar ret void automáticamente.

También validé que una función void no pueda usarse como expresión. Esto significa que una llamada a una función void es válida como statement, pero no puede asignarse a una variable ni formar parte de una operación aritmética.

9.7 Conversiones de tipos

La conversión implícita principal que implementé fue int a float. Esta conversión se permite porque no pierde la parte decimal de un valor existente; simplemente transforma un entero a su representación flotante. La instrucción LLVM usada para esto fue sitofp.

Centralicé las conversiones en funciones como cast_value y cast_assignment_value. Esto evitó repetir la misma lógica en asignaciones, declaraciones, llamadas a funciones y retornos. Si los tipos son iguales, el valor se deja igual; si se necesita int a float, se convierte; en otros casos se lanza un error.

No implementé conversiones automáticas de float a int, char a int o bool a int dentro del lenguaje general. Aunque algunas podrían tener sentido en C, decidí mantenerlas explícitas o limitarlas a casos especiales como printf para evitar comportamientos inesperados.

X. Implementación de Estructuras de Control

Las estructuras de control fueron esenciales para que el lenguaje pudiera representar programas más completos. En esta etapa implementé condicionales, ciclos y selección múltiple, traduciendo cada construcción a bloques básicos de LLVM conectados por saltos.

El documento de LLVM IR para estructuras de control muestra que un ciclo while se construye con bloques separados para condición, cuerpo y salida. Ese mismo principio lo apliqué en while, for, do/while, if/else y switch, adaptando la cantidad de bloques según la estructura.

La generación de estructuras de control también exigió validar que las condiciones fueran booleanas. Esto fue importante porque un salto condicional en LLVM necesita un i1, no un entero ni un flotante.

10.1 if

La estructura if evalúa una condición y ejecuta una sentencia o bloque solo si esa condición es verdadera. En el AST, la condición se guarda como una expresión y la rama verdadera como un conjunto de Statements.

Para generar LLVM, primero evalúo la condición y verifico que sea bool. Después creo un bloque if.then y un bloque if.end. Con builder.cbranch conecto el bloque actual hacia if.then si la condición es verdadera o hacia if.end si es falsa.

Cuando termina la rama verdadera, si el bloque no terminó con return, genero un branch hacia if.end. Esto permite que el flujo continúe después del if. Esta estructura fue la base para después implementar if/else.

10.2 if/else

El if/else extiende el if agregando una segunda rama para el caso falso. En el AST, el nodo IfStatement puede guardar else_stmts como None o como una lista de sentencias dependiendo de si existe else.

En LLVM genero tres bloques principales: if.then, if.else e if.end. La condición decide si el flujo entra al bloque then o al bloque else. Ambas ramas, si no terminan con return, saltan al bloque de unión if.end.

También contemplé el caso en el que ambas ramas terminan con return. En ese escenario, el bloque de unión puede quedar sin instrucciones útiles, por lo que se marca como unreachable para mantener válido el flujo de control.

10.3 while

El while se implementó creando tres bloques: while.cond, while.body y while.end. Primero se genera un salto incondicional hacia while.cond para evaluar la condición antes de ejecutar el cuerpo.

Dentro de while.cond se evalúa la expresión condicional. Si el resultado es true, el flujo salta a while.body. Si es false, salta a while.end. Esta estructura refleja el comportamiento del while: puede ejecutarse cero o más veces.

Al final del cuerpo, si no hubo return, el compilador genera un branch de regreso a while.cond. Esto permite repetir el ciclo. Esta implementación coincide con el modelo de bloques mostrado en el material de LLVM IR para estructuras de control.

10.4 do/while

El do/while se diferencia del while porque el cuerpo se ejecuta al menos una vez. Para implementarlo generé bloques do.body, do.cond y do.end. El flujo entra primero al bloque del cuerpo sin evaluar la condición inicialmente.

Después de ejecutar el cuerpo, si no terminó con return, el flujo salta a do.cond. En ese bloque se evalúa la condición. Si es verdadera, el flujo regresa al cuerpo; si es falsa, continúa en do.end.

Esta estructura permitió probar ciclos donde la actualización ocurre antes de revisar la condición. También ayudó a reforzar la importancia de posicionar correctamente el builder en cada bloque antes de emitir instrucciones.

10.5 for

El for se implementó separando sus partes en inicialización, condición, actualización y cuerpo. En el AST, ForStatement almacena init, condition, update y stmts. Esta representación facilitó generar cada parte en el orden correcto.

En LLVM se crean los bloques for.cond, for.body, for.update y for.end. Primero se ejecuta la inicialización, luego se salta a la condición. Si la condición es verdadera, se entra al cuerpo; después se ejecuta la actualización y se regresa a la condición.

Esta implementación permitió programas como ciclos de sumatoria, llenado de arreglos y cálculo de series. Aunque el for actual usa asignaciones simples en init y update, fue suficiente para cubrir los ejemplos requeridos.

10.6 switch/case/default

La estructura switch/case/default se implementó usando la instrucción switch de LLVM. El AST guarda la expresión evaluada, una lista de SwitchCase y un default opcional. Por simplicidad, el compilador acepta switch sobre expresiones de tipo int.

Para cada case se crea un bloque switch.case.valor. También se crea un bloque switch.default si existe default y un bloque switch.end para continuar después del switch. La instrucción switch conecta el valor evaluado con cada caso posible.

El break se implementó usando una pila de bloques de salida. Cuando el compilador entra a un switch, agrega switch.end a break_blocks. Si encuentra break, genera un branch hacia ese bloque. Al terminar el switch, elimina ese bloque de la pila.

10.7 break

El break se implementó inicialmente para switch. Su función es terminar el caso actual y saltar al final de la estructura. Para lograrlo, el IRGenerator mantiene una lista llamada break_blocks.

Cuando se genera un switch, el bloque switch.end se agrega a break_blocks. Cuando visit_break se ejecuta, revisa que exista un bloque disponible. Si no existe, lanza un error porque break no debería usarse fuera de switch.

Esta implementación podría extenderse después para while, for y do/while. Sin embargo, en el alcance actual se usó principalmente para switch/case, que era la estructura donde más se necesitaba evitar que el flujo continuara hacia otras instrucciones.

XI. Implementación de Funciones

Las funciones hicieron que el compilador pudiera manejar programas más cercanos a C real. Antes de implementarlas, el lenguaje podía trabajar principalmente dentro de main; después, fue posible definir varias funciones, pasar parámetros, retornar valores y llamar funciones desde otras funciones.

La implementación de funciones se basó en la gramática extendida de CLite, donde un programa puede tener funciones antes de main, parámetros, llamadas y retornos. Mi compilador amplía esta idea con funciones void, llamadas como statements y recursión.

A nivel de LLVM, cada función se representa con ir.Function y una firma definida por ir.FunctionType. La tabla de funciones fue esencial para que todas las llamadas pudieran resolverse antes de generar el cuerpo completo de cada función.

11.1 Funciones además de main

El compilador permite definir funciones además de main. Esto fue necesario para probar programas como factorial, Fibonacci, torres de Hanoi, serie de Taylor y juegos de palabras. Cada función se reconoce como un ProgramItem y se almacena dentro de tree.functions.

Durante la primera pasada, todas las funciones se registran en function_table. Esto incluye su nombre, objeto LLVM, tipo de retorno y tipos de parámetros. Esta tabla funciona como una declaración previa de todas las funciones disponibles.

También validé que exista una función main. Aunque el parser permite varias funciones, el programa necesita main como punto de entrada. Si main no existe, compile_source lanza un error indicando que el programa debe tener una función main.

11.2 Parámetros de función

Los parámetros se reconocen en la gramática mediante `ParameterList` y `Parameter`. Cada parámetro tiene un tipo y un identificador. En el AST se almacenan como objetos `Parameter` dentro de `FunctionDefinition`.

En la generación de LLVM, los parámetros se crean como parte de la firma de la función. Después, al generar el cuerpo, se copian a variables locales con `alloca` y `store`. Esto permite leerlos con `load` y tratarlos como variables normales.

Esta decisión también permitió que los parámetros participaran en expresiones, comparaciones, llamadas recursivas y asignaciones indirectas. En los ejemplos de torres de Hanoi, los parámetros `char` e `int` se pasan correctamente entre llamadas recursivas.

11.3 Retornos con valor

Las funciones que no son void deben retornar un valor. El nodo `Return` puede almacenar una expresión, y `visit_return` evalúa esa expresión antes de generar `builder.ret`.

Antes de emitir el retorno, el compilador valida que el tipo de la expresión coincida con el tipo de retorno de la función. Si la función retorna `float` y la expresión produce `int`, se aplica conversión implícita. En otros casos incompatibles se lanza un error.

También agregué una validación al final de cada función. Si una función no void termina sin un `return` en todos los caminos, el compilador lanza un `TypeError`. Esto evita generar funciones incompletas que no devuelven el valor prometido por su firma.

11.4 Funciones void

Las funciones void fueron implementadas para permitir procedimientos que realizan acciones sin regresar un valor. Esto fue necesario para ejemplos como `printWord`, `touch`, `countdown` y `towerOfHanoi`.

En el parser permití `return` sin expresión. En el `IRGenerator`, si la función actual es void y el nodo `Return` no tiene expresión, se genera `ret void`. Si una función void intenta retornar un valor, el compilador reporta un error.

También permití que las funciones void terminen sin un `return` explícito. Si al finalizar la generación del cuerpo el bloque no está terminado, `compile_source` agrega `ret void` automáticamente. Esto hace que funciones simples como void `printWord` sean válidas aunque no tengan `return` final.

11.5 Llamadas generales a funciones

Las llamadas generales permiten usar cualquier función declarada en el programa, no solo `printf`. El parser construye un nodo `Call` con el nombre de la función y la lista de argumentos.

Durante la generación de código, `emit_call` busca la función en `function_table`. Si no existe, lanza un `NameError`. Esto permitió detectar errores causados por funciones mal declaradas, tipos no soportados o problemas de parsing que impedían registrar una función.

También se validó el número de argumentos y se aplicaron conversiones cuando el tipo esperado lo permite. Así, una función que espera `float` puede recibir un `int` y convertirlo antes de generar la llamada LLVM.

11.6 Recursión

La recursión fue posible gracias al diseño de dos pasadas. Como la firma de cada función se registra antes de generar su cuerpo, una función puede llamarse a sí misma dentro de su implementación.

Esto se probó con ejemplos como Fibonacci y torres de Hanoi. En Fibonacci, la función fib llama a fib(n - 1) y fib(n - 2). En torres de Hanoi, la función towerOfHanoi realiza dos llamadas recursivas cambiando los parámetros de origen, destino y auxiliar.

La recursión también sirvió como prueba fuerte del manejo de parámetros y retornos. Cada llamada crea su propio contexto de ejecución en tiempo de ejecución, mientras que el compilador solo necesita generar correctamente la instrucción call con los argumentos adecuados.

XII. Implementación de Arreglos

Los arreglos fueron una de las extensiones más importantes del compilador. Permitieron trabajar con colecciones de valores de tamaño fijo y probar programas que requieren listas simples, como secuencias para juegos de palabras o acumuladores por índice.

La implementación se limitó a arreglos locales de una dimensión. Esto fue suficiente para cubrir lectura, escritura, inicialización por defecto y acceso mediante índices. La representación en LLVM se hizo con ArrayType y getelementptr.

Aunque el compilador ya acepta sintaxis de declaraciones globales, la generación completa de arreglos globales y arreglos como parámetros quedó como trabajo futuro. Aun así, los arreglos locales funcionan correctamente para los casos probados.

12.1 Declaración de arreglos

La declaración de arreglos se agregó a la gramática mediante Declarator : ID [INTLIT]. Esto permite escribir declaraciones como int arr[5]; dentro de funciones o bloques.

En el AST, la declaración del arreglo se representa con Declaration y el atributo array_size. Si array_size tiene un valor entero, el IRGenerator sabe que debe crear un arreglo y no una variable simple.

En LLVM, el arreglo se crea con ir.ArrayType. Por ejemplo, un arreglo de cinco enteros se representa como [5 x i32]. Después se reserva memoria para todo el arreglo con alloca.

12.2 Inicialización por defecto

Cuando se declara un arreglo, el compilador inicializa cada elemento con el valor por defecto de su tipo. Esto se hizo para evitar lecturas de memoria no inicializada.

La inicialización se genera recorriendo cada índice del arreglo. Para cada posición se crea un getelementptr y luego un store con el valor por defecto. Aunque esto genera varias instrucciones, hace explícito el estado inicial del arreglo.

Esta decisión fue consistente con la inicialización por defecto de variables normales. En los ejemplos, permitió usar arreglos con valores conocidos incluso antes de asignar manualmente cada posición.

12.3 Lectura de elementos

La lectura de elementos se realiza mediante `ArrayAccess` cuando aparece dentro de una expresión. El compilador evalúa el índice, obtiene la dirección del elemento con `getelementptr` y luego genera un `load`.

El resultado del `load` se guarda en la pila temporal con el tipo del elemento. Esto permite usar `arr[i]` dentro de sumas, asignaciones, llamadas a funciones o retornos.

También se valida que el índice sea `int`. Si el índice produce `float`, `bool`, `char` o `string`, el compilador lanza un error. Esto simplifica la semántica y evita accesos inválidos.

12.4 Escritura de elementos

La escritura de elementos ocurre cuando `ArrayAccess` aparece del lado izquierdo de una asignación. En este caso, el compilador no carga el valor, sino que obtiene la dirección del elemento para poder almacenar ahí el resultado.

Después de obtener la dirección, se evalúa la expresión del lado derecho y se valida su tipo. Si el arreglo es `float` y se asigna un `int`, se permite la conversión. Si el tipo es incompatible, se lanza un error.

Esto permitió programas como `arr[i] = inc(i);` y `total = total + arr[i];`. Estas pruebas confirmaron que el compilador podía combinar arreglos, llamadas a funciones, ciclos y expresiones aritméticas.

12.5 Validaciones semánticas

Agregué validaciones para detectar usos incorrectos de arreglos. Por ejemplo, si una variable normal se usa como arreglo, el compilador lanza un error indicando que no es un arreglo.

También evité asignar directamente al arreglo completo. Una instrucción como `arr = 10` no es válida en la implementación actual, porque solo se permite escribir elementos específicos mediante `arr[indice]`.

Otra validación fue exigir que el tamaño del arreglo sea mayor que cero. Si se declara `int arr[0]`, el compilador reporta que el tamaño debe ser mayor que cero. Esto evita crear arreglos sin espacio útil.

12.6 Limitaciones actuales de arreglos

La principal limitación actual es que no se soportan arreglos como parámetros. Esto significa que una firma como `void f(int arr[5])` no se genera correctamente. LLVM requiere representar este caso como un puntero, no como un arreglo local.

Tampoco se implementaron inicializadores con llaves, como `int arr[3] = {1, 2, 3};`. Actualmente se debe declarar el arreglo y asignar cada posición por separado. Esta decisión mantuvo la gramática y la generación de código más simples.

Finalmente, no hay verificación de rango en tiempo de ejecución. Si el índice es una expresión variable, el compilador no comprueba si está dentro del tamaño del arreglo. Para índices literales podría agregarse una validación de compilación como mejora futura.

XIII. Carga y Compilación de Ejemplos

Para validar el compilador con programas reales, organicé varios archivos fuente dentro de la carpeta Examples. Esto permitió separar los casos de prueba del código del compilador y ejecutar la misma función de compilación sobre diferentes programas.

La actividad solicitaba incluir al menos cinco programas de ejemplo con llamadas recursivas, como factorial, Fibonacci y serie de Taylor. Además de esos casos, agregué pruebas con torres de Hanoi, switch/case, sumatoria, juegos de palabras y otros programas que combinan funciones, ciclos y printf.

Esta etapa fue útil porque los errores no siempre aparecen con programas pequeños. Al correr todos los ejemplos, pude detectar problemas como funciones no declaradas, strings globales duplicados, tipos no soportados y limitaciones con arreglos como parámetros.

13.1 Lectura de archivos desde Examples

Implementé una forma de leer los archivos .c desde la carpeta Examples usando pathlib. Esto permitió cargar el contenido del archivo como texto y enviarlo directamente a compile_source.

La lectura desde archivos hizo que el flujo fuera más parecido al uso real de un compilador. En lugar de cambiar manualmente una cadena de datos dentro del código, pude seleccionar un archivo fuente y compilarlo.

También pude recorrer automáticamente todos los archivos .c de la carpeta. Esto sirvió como una prueba general para verificar que los ejemplos siguieran siendo compatibles después de modificar la gramática o el IRGenerator.

13.2 Función de compilación general

La función compile_source concentra el proceso completo de compilación. Recibe el código fuente como texto, crea el lexer, crea el parser, construye el AST y luego genera el módulo LLVM IR.

Dentro de esta función se configuran los tipos LLVM, se crea el módulo y se ejecutan las dos pasadas de funciones. Esto evitó tener el flujo de compilación repetido en varias partes del archivo.

Al final, compile_source retorna el módulo LLVM generado. Esto permite imprimirlo, guardarlo en un archivo .ll o eventualmente ejecutarlo con MCJIT. Esta función se convirtió en la entrada principal del compilador.

```
def compile_source(data: str):
    lexer = lex.lex()
    parser = yacc.yacc()
    tree = parser.parse(data, lexer=lexer)
    if tree is None:
        raise SyntaxError('El parser no pudo construir el AST.')
    ...
    return module
```

13.3 Validación de múltiples ejemplos

Para validar múltiples ejemplos, recorrí la carpeta Examples y compilé cada archivo. En cada iteración se imprime el nombre del archivo para saber cuál se está procesando.

Este flujo ayudó a identificar rápidamente qué ejemplo fallaba cuando aparecía un error. Por ejemplo, si un archivo usaba double, el compilador reportaba que la función correspondiente no se había declarado porque double no estaba soportado por la gramática.

La validación por lotes fue importante porque el compilador creció de forma incremental. Cada nueva característica podía afectar reglas anteriores, por lo que ejecutar todos los ejemplos ayudó a detectar regresiones.

13.4 Generación de LLVM IR por ejemplo

Después de compilar cada ejemplo, el módulo LLVM se puede imprimir para revisar la salida. Esta salida muestra funciones, bloques, instrucciones load, store, branch, call y returns.

Revisar el IR fue una parte importante de la depuración. Por ejemplo, en torres de Hanoi pude verificar que los parámetros char se representaran como i8 y que antes de printf se extendieran a i32 con zext.

También pude validar que los ciclos generaran bloques correctos, que los switch generaran la instrucción switch de LLVM y que las llamadas recursivas se emitieran como call a la misma función.

XIV. Pruebas Realizadas

Las pruebas fueron realizadas con programas pequeños y programas completos. La intención fue confirmar que cada característica funcionara de forma aislada y también combinada con otras características del compilador.

Además de revisar que el parser aceptara los programas, también revisé el LLVM IR generado. Esto fue importante porque un programa puede parsearse correctamente, pero generar instrucciones inválidas si la semántica o los tipos no se manejan bien.

Los ejemplos usados cubren operaciones aritméticas, condicionales, ciclos, funciones recursivas, arreglos, switch/case, printf y programas completos. Estos casos cumplen con el objetivo de demostrar la capacidad del compilador para manejar construcciones representativas.

14.1 Pruebas con operaciones aritméticas

Probé operaciones como suma, resta, multiplicación, división y módulo con enteros. Estas pruebas permitieron validar que BinaryOp generara add, sub, mul, sdiv y srem según el operador.

También probé operaciones con float y combinaciones de int con float. En esos casos revisé que el compilador generara sitofp antes de la operación flotante y que usara instrucciones como fadd o fdiv.

Estas pruebas fueron importantes porque las expresiones aritméticas aparecen en casi todos los demás ejemplos: ciclos, retornos, llamadas recursivas, sumatorias, series y actualizaciones de variables.

14.2 Pruebas con condicionales

Las pruebas con condicionales incluyeron if simple, if/else y condiciones compuestas. Verifiqué que el parser aceptara tanto bloques como sentencias individuales después del if.

En el IR revisé la creación de bloques if.then, if.else e if.end. También confirmé que las ramas con return terminaran correctamente y que no se generaran saltos innecesarios después de bloques ya terminados.

Estas pruebas fueron necesarias para ejemplos recursivos como Fibonacci, donde los casos base dependen de if. Si el if no funciona correctamente, la recursión no puede detenerse de forma segura.

14.3 Pruebas con ciclos

Probé ciclos while, do/while y for. En while validé que la condición se evaluara antes del cuerpo. En do/while validé que el cuerpo se ejecutara antes de revisar la condición. En for validé inicialización, condición, cuerpo y actualización.

Los ejemplos de factorial, sumatoria y serie de Taylor ayudaron a probar ciclos con acumuladores. Estos casos combinan variables locales, comparaciones, operaciones aritméticas y asignaciones repetidas.

En el IR revisé que los ciclos generaran bloques de condición, cuerpo, actualización y salida según correspondiera. Esta revisión me permitió confirmar que los branches regresaban al bloque correcto.

14.4 Pruebas con funciones recursivas

Las funciones recursivas fueron una prueba importante porque dependen de que la función esté registrada antes de generar su cuerpo. Probé Fibonacci y torres de Hanoi para validar este comportamiento.

En Fibonacci revisé llamadas como fib(n - 1) y fib(n - 2). En torres de Hanoi revisé llamadas recursivas con varios parámetros, incluyendo chars que representan las torres.

Estas pruebas demostraron que function_table estaba funcionando correctamente y que los parámetros se copiaban a memoria local para ser usados dentro de cada llamada.

14.5 Pruebas con arreglos

Probé declaraciones de arreglos, asignaciones por índice y lecturas dentro de expresiones. Un caso básico fue llenar un arreglo con valores y retornar la suma de varias posiciones.

También probé arreglos dentro de ciclos for, usando una variable como índice. Esto permitió verificar que get_array_element_ptr aceptara índices generados por expresiones y no solo literales.

Los arreglos también se usaron para construir ejemplos como una secuencia de palabras. En esos casos, el arreglo almacena códigos enteros y otra función usa switch para decidir qué palabra imprimir.

14.6 Pruebas con switch/case

Las pruebas con switch/case incluyeron casos con varios case, default y break. Verifiqué que el switch aceptara expresiones enteras y que cada case generara un bloque diferente.

También probé switch combinado con funciones. Por ejemplo, una función printWord recibe un entero y usa switch para imprimir Choco, La o Te según el valor.

El IR generado usa la instrucción switch de LLVM. Esto permitió confirmar que el compilador no estaba simulando el switch con if anidados, sino usando una instrucción propia de LLVM para selección múltiple.

14.7 Pruebas con printf

Probé printf con strings, enteros, floats, chars y llamadas a funciones como argumentos. Esto permitió validar la generación de strings globales y la llamada variádica a printf.

También corregí el problema de nombres duplicados en strings globales. Este error apareció cuando funciones distintas generaban literales string y cada IRGenerator intentaba crear .str0.

Las pruebas con printf fueron importantes porque dieron una forma visible de revisar resultados, más allá de inspeccionar el valor retornado por main.

14.8 Pruebas con ejemplos completos

Finalmente probé programas completos dentro de la carpeta Examples. Entre ellos se encuentran factorial, Fibonacci, torres de Hanoi, sumatoria, serie de Taylor, calculadora con switch y juego de palabras.

Estos ejemplos combinan múltiples características: funciones, llamadas, recursión, ciclos, arreglos, printf, condiciones y tipos distintos. Por eso fueron más representativos que pruebas aisladas.

El hecho de que estos programas generaran LLVM IR permitió comprobar que el compilador ya podía procesar programas de tamaño moderado y no solo fragmentos pequeños de código.

XV. Código Clave del Proyecto

En esta sección se presentan fragmentos clave del proyecto. El objetivo no es repetir todo el código fuente, sino explicar las partes que hacen posible el funcionamiento del compilador.

El código clave se encuentra principalmente en analysis.py y arbol.py. El primero contiene el lexer, el parser y la función general de compilación. El segundo contiene las clases del AST, el Visitor y el IRGenerator.

Estos fragmentos muestran cómo se conecta cada etapa: tokens, reglas gramaticales, nodos del AST y generación de LLVM IR. También permiten entender las decisiones de diseño más importantes del proyecto.

15.1 Código clave del lexer

El lexer define los tokens, palabras reservadas y reglas para literales. Esta parte es clave porque todos los errores o aciertos del parser dependen de que el lexer entregue correctamente la secuencia de tokens.

Uno de los fragmentos más importantes es t_ID, porque distingue entre identificadores normales y palabras reservadas. También convierte true y false a valores booleanos de Python.

Otro fragmento importante es el manejo de strings y chars, porque estos valores requieren interpretar comillas y secuencias de escape. Sin estas reglas, printf y char no funcionarían correctamente.

```
def t_ID(t):
    r'[a-zA-Z_][a-zA-Z_0-9]*'
    lower_value = t.value.lower()
    t.type = reserved.get(lower_value, 'ID')
    if t.type == 'TRUE':
        t.value = True
    elif t.type == 'FALSE':
        t.value = False
    return t
```

15.2 Código clave del parser

El parser es clave porque define qué programas acepta el lenguaje. La regla `Statement` concentra la mayoría de las instrucciones soportadas, como asignaciones, retornos, `if`, `switch`, ciclos, llamadas, bloques y sentencias vacías.

La gramática de expresiones también fue fundamental. Al separar `Expression`, `Conjunction`, `Equality`, `Relation`, `Addition`, `Term`, `Factor` y `Primary`, pude representar correctamente la precedencia de operadores.

Además, las reglas de funciones y `ProgramItemTail` permitieron diferenciar entre funciones y declaraciones globales sin provocar errores de ambigüedad en PLY.

```
def p_Statement(p):
    """
    Statement : Assignment
              | Return
              | Print
              | IfStatement
              | SwitchStatement
              | Break
              | WhileStatement
              | DoWhileStatement
              | ForStatement
              | CallStatement
              | BlockStatement
              | EmptyStatement
    """
    p[0] = p[1]
```

15.3 Código clave del AST

El AST se basa en clases que representan elementos del lenguaje. La clase `Program` separa funciones y globales, mientras que `FunctionDefinition` representa cada función con su tipo de retorno, nombre, parámetros, declaraciones y sentencias.

Los nodos de expresiones como `BinaryOp`, `CompareOp` y `LogicalOp` permiten construir árboles para expresiones complejas. Esto hizo posible que el `IRGenerator` no dependiera directamente de la gramática concreta.

Los nodos como `ArrayAccess`, `CallStatement` y `BlockStatement` surgieron conforme el lenguaje creció. Cada uno resolvió una necesidad concreta: acceso a arreglos, llamadas void y bloques con scope local.

```
class FunctionDefinition(ASTNode):
    def __init__(self, return_type, name, params, decls, stmts):
        self.return_type = return_type
        self.name = name
        self.params = params
        self.decls = decls
        self.stmts = stmts

    def accept(self, visitor):
        visitor.visit_function_definition(self)
```

15.4 Código clave del IRGenerator

El `IRGenerator` es la parte que traduce el AST a LLVM IR. Sus métodos `visit_*` corresponden directamente con los nodos del AST. Esto aplica el patrón `Visitor` y mantiene separada la representación del programa de su generación de código.

Dentro del constructor se inicializan estructuras como `stack`, `symbol_table`, `scope_stack`, `break_blocks` y `functions`. Estas estructuras permiten evaluar expresiones, recordar variables, controlar scopes, resolver break y llamar funciones.

Los métodos más importantes son `visit_declaration`, `visit_assignment`, `visit_binary_op`, `visit_if_statement`, `visit_while_statement`, `visit_for_statement`, `visit_switch_statement`, `emit_call` y `visit_return`. Estos cubren la mayoría del comportamiento del lenguaje.

```
class IRGenerator(Visitor):
    def __init__(self, builder, intType, floatType, stringType,
                 boolType, charType, module, return_type, functions=None):
        self.stack = []
        self.symbol_table = {}
        self.scope_stack = [set()]
        self.builder = builder
        self.functions = functions if functions is not None else {}
```

15.5 Código clave de la función `compile_source`

La función `compile_source` une todas las etapas del compilador. Recibe código fuente, crea el lexer y parser, construye el AST y genera el módulo LLVM.

Dentro de esta función se crean los tipos LLVM y se hacen las dos pasadas sobre las funciones. La primera pasada registra firmas; la segunda genera cuerpos. Esta estructura fue clave para permitir recursión.

Al regresar el módulo LLVM, la función permite imprimir el IR, guardarlo o ejecutarlo posteriormente. Por eso `compile_source` funciona como la interfaz principal del compilador.

```
def compile_source(data: str):
    lexer = lex.lex()
    parser = yacc.yacc()
    tree = parser.parse(data, lexer=lexer)
    if tree is None:
        raise SyntaxError('El parser no pudo construir el AST.')
    ...
    return module
```

15.6 Código clave de los ejemplos

Los ejemplos fueron clave porque demostraron que el compilador funciona con programas completos. No solo prueban reglas individuales, sino combinaciones de funciones, ciclos, condiciones, arreglos y `printf`.

Programas como `hanoiTower.c` prueban funciones void, parámetros char, recursión y `printf`. Programas como `taylorSerie.c` prueban ciclos for, floats y operaciones aritméticas. Juegos de palabras prueban switch, arreglos y llamadas entre funciones.

Estos ejemplos también sirvieron para descubrir errores. Por ejemplo, los strings duplicados aparecieron al compilar programas con `printf` en varias funciones, y el uso de `double` mostró una limitación actual del lenguaje.

XVI. Problemas Encontrados y Soluciones

Durante el desarrollo aparecieron varios problemas típicos de un compilador: errores de gramática, conflictos en PLY, validación de tipos, funciones no registradas, nombres duplicados en LLVM y limitaciones de arreglos.

Estos problemas fueron útiles porque mostraron que implementar un compilador no consiste solo en aceptar sintaxis, sino en mantener coherencia entre el lexer, parser, AST, tabla de símbolos y generador de código.

Cada solución fue incorporada de forma incremental. En muchos casos, corregir un error implicó ajustar más de una etapa, porque un cambio en la gramática podía requerir cambios en el AST y en el IRGenerator.

16.1 Errores de gramática en PLY

Uno de los problemas más comunes fue que PLY no pudiera construir el parser. Esto sucedía cuando una regla usaba un símbolo no definido, cuando una regla quedaba inaccesible o cuando había recursión infinita.

Un ejemplo fue el error donde `Call` se usaba en `CallStatement` y `Primary`, pero la regla `p_Call` no estaba definida. Como consecuencia, `OptionalArguments` quedaba como regla inalcanzable y PLY reportaba errores al construir el parser.

La solución fue revisar la cadena completa de reglas y agregar la producción faltante. Este tipo de error reforzó la importancia de que cada no terminal usado en una regla tenga una definición clara.

16.2 Conflictos shift/reduce

Aparecieron conflictos shift/reduce, especialmente con el caso clásico del dangling else. Este problema ocurre cuando el parser debe decidir si un else pertenece al if más cercano o a uno más externo.

Para resolverlo, agregué precedencia con `IFX` y `ELSE`. Después apliqué `%prec IFX` en la regla del if sin else. Con esto, PLY puede asociar correctamente el else al if más cercano.

Este problema mostró que una gramática puede parecer correcta visualmente, pero aun así ser ambigua para un parser LALR. Resolverlo ayudó a hacer el análisis sintáctico más estable.

16.3 Problemas con funciones no declaradas

En varias ocasiones aparecieron errores indicando que una función no había sido declarada. No siempre significaba que faltara la función; a veces el parser no la había reconocido correctamente por un tipo no soportado o una firma inválida.

Un caso específico ocurrió con una función `double e(int x, int n)`. Como `double` no estaba definido en el lexer ni en la gramática, la función no entraba a `function_table` y después la llamada `e(x, n)` fallaba.

La solución en ese caso fue adaptar el ejemplo a `float` o implementar soporte completo para `double`. Esto mostró que los errores semánticos pueden originarse en etapas anteriores como el análisis léxico o sintáctico.

16.4 Problemas con nombres duplicados en strings globales

Al compilar programas con varios `printf` en funciones diferentes, apareció `DuplicatedNameError` con nombres como `.str0`. Esto ocurrió porque cada IRGenerator iniciaba `string_count` en cero.

La solución fue modificar `create_global_string` para revisar `module.globals` antes de crear una variable global. Si el nombre ya existía, el contador avanzaba hasta encontrar uno libre.

Este problema fue importante porque mostró que algunas estructuras deben ser únicas a nivel de módulo, no solo dentro de una función. Los strings globales pertenecen al módulo completo y por eso sus nombres no pueden repetirse.

16.5 Problemas con tipos no soportados

El compilador soporta int, float, bool, char, string y void, pero no soporta double. Algunos ejemplos escritos en C real usan double, lo cual produjo errores durante el parsing o la generación de funciones.

Para resolverlo a corto plazo, adapté los ejemplos a float. Esto permitió mantener el alcance del proyecto sin rediseñar todas las operaciones de punto flotante.

A largo plazo, se podría implementar double agregando DOUBLE al lexer, a Type, a get_llvm_type, a visit_literal, a las conversiones y a las operaciones flotantes. Es una mejora posible, pero requiere consistencia en varias etapas.

16.6 Problemas con arreglos como parámetros

Intenté usar funciones con parámetros tipo int sequence[15], pero el compilador no soportaba arreglos como parámetros. El parser podía fallar o el IRGenerator no podía representarlos correctamente.

La razón es que un arreglo local y un arreglo como parámetro no se representan igual en LLVM. Un arreglo local se reserva como [N x tipo], mientras que un arreglo como parámetro normalmente debe tratarse como un puntero al primer elemento.

La solución temporal fue usar arreglos locales dentro de la función o evitar pasar arreglos como parámetros. Como mejora futura, se podría extender Parameter para guardar array_size y generar punteros adecuados.

XVII. Limitaciones Actuales

Aunque el compilador cubre muchas construcciones importantes, todavía tiene limitaciones. Estas limitaciones no impiden demostrar el funcionamiento del proyecto, pero sí marcan áreas donde el compilador podría crecer.

Las limitaciones principales se relacionan con variables globales, arreglos como parámetros, inicializadores de arreglos, validación de rangos, soporte para double y optimización del código generado.

Documentar estas limitaciones es importante porque permite distinguir entre errores y decisiones de alcance. Algunas características fueron dejadas fuera para mantener el proyecto manejable y concentrarse en las etapas principales del compilador.

17.1 Variables globales aceptadas pero no generadas completamente en LLVM

La gramática ya puede aceptar declaraciones globales y el AST las almacena en Program.globals. Sin embargo, la generación completa de LLVM para estas variables todavía no está implementada.

Esto significa que el parser puede reconocer una declaración global, pero si el programa intenta usarla dentro de una función, el IRGenerator no la encuentra en symbol_table local.

Para soportarlas completamente, sería necesario crear `GlobalVariable` en el módulo LLVM y modificar `visit_variable` y `visit_assignment` para buscar primero en variables locales y luego en globales.

17.2 Arreglos como parámetros

Los arreglos como parámetros no están soportados actualmente. Una firma como `void f(int arr[5])` requiere un tratamiento diferente al de una declaración local.

En LLVM, un arreglo pasado como parámetro normalmente se maneja como un puntero. Esto implica cambiar la representación de `Parameter` y la lógica de `get_array_element_ptr` para distinguir arreglos locales y arreglos recibidos.

Por ahora, los ejemplos que necesitan secuencias usan arreglos locales dentro de la función o evitan pasar arreglos entre funciones. Esta limitación está claramente identificada como mejora futura.

17.3 Inicializadores de arreglos

El compilador no soporta inicializadores con llaves, como `int arr[3] = {1, 2, 3};`. Actualmente los arreglos se declaran y luego se asignan elemento por elemento.

Esta limitación simplificó el parser y el `IRGenerator`. Implementar inicializadores requeriría una nueva regla para listas de expresiones y una validación para comparar la cantidad de elementos con el tamaño declarado.

Aunque es una característica útil, no fue necesaria para demostrar lectura, escritura y uso de arreglos en ciclos o funciones. Por eso quedó fuera del alcance principal.

17.4 Verificación de rango en arreglos

Actualmente el compilador valida que el índice de un arreglo sea `int`, pero no verifica si está dentro del rango válido. Por ejemplo, `arr[10]` podría generar IR incluso si `arr` tiene tamaño 5.

Para índices literales, sería posible detectar el error en compilación comparando el valor con `array_size`. Para índices calculados en ejecución, se requerirían checks adicionales en LLVM IR.

Esta limitación es común en compiladores simples. La implementación actual genera `getelementptr inbounds`, pero no agrega una rutina de error o validación dinámica de límites.

17.5 Soporte incompleto para double

El tipo `double` no está soportado actualmente. El compilador maneja `float` como tipo de punto flotante, lo cual fue suficiente para los ejemplos adaptados.

Agregar `double` requeriría cambios en el lexer, parser, AST, generación de tipos, literales, operaciones, conversiones y `printf`. No basta con agregar una palabra reservada; debe integrarse en todas las etapas.

Por esta razón, los ejemplos que originalmente usaban `double` fueron modificados a `float`. Esta decisión mantuvo el compilador consistente y evitó generar código parcial o incorrecto.

17.6 Optimización de código generado

El LLVM IR generado es correcto para los casos probados, pero no está optimizado. Por ejemplo, algunas variables se inicializan por defecto y luego se vuelven a asignar inmediatamente.

También se generan cargas y stores explícitos para muchas operaciones. Esto hace que el IR sea más largo, pero también más fácil de leer y depurar durante el desarrollo del compilador.

Una mejora futura sería pasar el módulo por optimizaciones de LLVM para eliminar stores redundantes, simplificar bloques y mejorar la eficiencia del código generado.

XVIII. Posibles Mejoras Futuras

El proyecto deja varias rutas claras para seguir creciendo. Muchas de las mejoras futuras corresponden a características que ya están parcialmente preparadas en la gramática o en el AST, pero que todavía no se completaron en la generación de código.

También hay mejoras relacionadas con calidad de implementación, como mejor manejo de errores, optimización del IR y ejecución directa del código generado. Estas mejoras harían que el compilador fuera más robusto y más cómodo de usar.

La ventaja de haber construido el compilador por etapas es que estas mejoras pueden agregarse de forma incremental. Cada nueva característica puede conectarse al lexer, parser, AST e IRGenerator sin tener que rehacer todo el proyecto.

18.1 Soporte completo para variables globales

Una mejora importante sería completar la generación de variables globales. El parser y el AST ya pueden reconocerlas, por lo que falta principalmente crear sus representaciones LLVM.

Esto implicaría generar `GlobalVariable` para cada declaración global y mantener una tabla de símbolos global separada de las variables locales. También habría que permitir cargas y stores sobre estas variables desde cualquier función.

Con esta mejora, el compilador podría aceptar programas que compartan estado entre funciones, como contadores globales, arreglos globales o constantes definidas fuera de `main`.

18.2 Arreglos como parámetros

Otra mejora sería permitir arreglos como parámetros de función. Esto permitiría escribir funciones que reciban listas sin tener que declararlas localmente dentro de cada llamada.

Para implementarlo correctamente, habría que representar el parámetro como un puntero al tipo del elemento. También sería necesario adaptar `ArrayAccess` para manejar tanto arreglos locales como punteros recibidos.

Esta mejora haría posibles ejemplos más naturales, como funciones que suman los elementos de un arreglo o que reciben una secuencia de códigos para imprimir palabras.

18.3 Inicializadores con llaves

Los inicializadores con llaves harían más cómodo trabajar con arreglos. En lugar de escribir varias asignaciones, se podría declarar `int arr[3] = {1, 2, 3};`.

Para implementarlo, el parser necesitaría una regla para listas de expresiones. El AST debería guardar esa lista dentro de `Declaration`, y el IRGenerator tendría que almacenar cada valor en el índice correspondiente.

También habría que validar que la cantidad de elementos no exceda el tamaño del arreglo y que todos los valores sean compatibles con el tipo del arreglo.

18.4 Soporte para double

Agregar double permitiría que los programas escritos en C real requirieran menos modificaciones. Muchos ejemplos de series matemáticas usan double para mayor precisión.

La implementación debería incluir un token DOUBLE, una regla en Type, representación con `ir.DoubleType` y operaciones flotantes compatibles. También habría que decidir reglas de conversión entre `int`, `float` y `double`.

Esta mejora ampliaría el rango de programas numéricos que el compilador puede procesar y haría más fieles ejemplos como la serie de Taylor para e^x .

18.5 Ejecución directa del código generado

Actualmente el compilador genera LLVM IR y lo imprime. Una mejora sería integrar ejecución directa usando MCJIT de `llvmlite.binding`.

Esto permitiría compilar y ejecutar `main` desde Python, obteniendo directamente el resultado del programa. También facilitaría crear pruebas automatizadas que comparen resultados esperados contra resultados reales.

La ejecución directa sería especialmente útil para ejemplos como factorial, Fibonacci, sumatoria y juegos de palabras, donde se puede verificar tanto el retorno como la salida de `printf`.

18.6 Mejor manejo de errores

Aunque el compilador ya detecta varios errores, el manejo podría ser más claro. Una mejora sería reportar errores con nombre de archivo, línea, columna y una descripción más amigable.

También sería útil separar errores léxicos, sintácticos y semánticos en clases distintas. Esto ayudaría a saber si el problema viene de un token inválido, una regla gramatical o una validación de tipos.

Otra mejora sería detener el parser inmediatamente ante errores sintácticos para evitar AST incompletos. Esto ya se identificó durante el desarrollo y puede reforzarse para todos los casos.

18.7 Optimización del LLVM IR

El IR actual prioriza claridad sobre eficiencia. Una mejora futura sería aplicar pases de optimización de LLVM para simplificar instrucciones y eliminar redundancias.

Por ejemplo, se podrían eliminar stores duplicados, cargas innecesarias o bloques que solo redirigen a otro bloque. Esto haría que el IR generado fuera más cercano al producido por compiladores maduros.

La optimización no fue necesaria para demostrar el funcionamiento del compilador, pero sería una etapa natural después de completar la generación correcta del IR.

XIX. Manual de Usuario

Esta sección explica cómo usar el compilador desde el proyecto. El objetivo es que otra persona pueda ubicar los archivos principales, colocar programas de prueba y generar LLVM IR a partir de ellos.

El manual se basa en la organización actual del proyecto, donde `analysis.py` contiene el lexer, parser y flujo de compilación, mientras que `arbol.py` contiene el AST y el IRGenerator. Los ejemplos se colocan dentro de la carpeta `Examples`.

Para usar el compilador correctamente, los programas fuente deben respetar los tipos y construcciones soportadas. Algunas características de C real, como `double` o arreglos como parámetros, deben evitarse o adaptarse a las capacidades actuales.

19.1 Requisitos y estructura de ejecución

Para ejecutar el compilador se requiere Python con las librerías `PLY` y `llvmlite` instaladas. `PLY` se usa para construir el lexer y el parser, mientras que `llvmlite` se usa para generar el código intermedio LLVM IR.

Los archivos principales deben mantenerse dentro de la carpeta `Compiler`. El archivo `analysis.py` importa `arbol.py`, por lo que ambos deben estar en el mismo contexto de ejecución o accesibles dentro del proyecto.

Los programas de prueba deben guardarse como archivos `.c` dentro de la carpeta `Examples`. Aunque pueden parecer programas de C, deben usar únicamente las características soportadas por el compilador.

19.2 Ejecución de ejemplos

Para compilar un ejemplo, se carga el archivo fuente y se pasa su contenido a `compile_source`. La función retorna un módulo LLVM que puede imprimirse con `print(module)`.

También se puede recorrer automáticamente la carpeta `Examples` y compilar todos los archivos `.c`. Esta forma es útil para verificar que los cambios al compilador no rompan ejemplos anteriores.

Si un ejemplo falla, el nombre del archivo impreso antes de compilar ayuda a localizar el problema. Después se puede revisar el mensaje de error para determinar si el fallo es léxico, sintáctico o semántico.

```
for example_path in sorted(EXAMPLES_DIR.glob('*.c')):
    data = example_path.read_text(encoding='utf-8')
    module = compile_source(data)
    print(module)
```

19.3 Interpretación de resultados

El resultado principal del compilador es el LLVM IR. Este IR muestra las funciones generadas, los bloques básicos, las instrucciones de memoria, las operaciones aritméticas, las comparaciones y las llamadas a funciones.

Para revisar si el código es correcto, se puede inspeccionar que cada función tenga su firma, que `main` exista, que las variables se creen con `alloca`, que las asignaciones usen `store` y que las estructuras de control generen bloques y branches apropiados.

En el caso de `printf`, el IR debe mostrar la declaración externa de `printf` y strings globales como `.str0`, `.str1`, etc. Si aparecen errores de nombres duplicados o funciones no declaradas, se debe revisar el generador de strings o la tabla de funciones.

XX. Reflexión Personal

Este proyecto fue una oportunidad para entender cómo funciona un compilador más allá de la teoría. Al implementar cada etapa, pude ver cómo una decisión en el lexer afecta al parser, cómo una regla gramatical afecta al AST y cómo cada nodo del AST necesita una traducción concreta a LLVM IR.

También aprendí que un compilador se construye mejor de forma incremental. Cada nueva característica debe probarse con casos pequeños antes de combinarse con el resto del lenguaje. Esta forma de trabajo me ayudó a resolver errores complejos sin perder de vista el flujo general.

La experiencia me permitió conectar conceptos de lenguajes de programación, estructuras de datos, análisis semántico, memoria, funciones, recursión y representación intermedia. El resultado fue un compilador pequeño, pero suficientemente completo para demostrar muchas ideas fundamentales.

20.1 Aprendizajes técnicos

Aprendí a construir un lexer con expresiones regulares y a usar PLY para convertir texto fuente en tokens. También entendí mejor cómo las palabras reservadas deben distinguirse de los identificadores comunes.

En el parser aprendí a diseñar reglas gramaticales, resolver ambigüedades y construir un AST. La reestructuración de expresiones para respetar precedencia fue una de las partes más importantes para comprender cómo se modela un lenguaje.

En la generación de código aprendí a usar llvmlite para crear funciones, bloques, variables, operaciones y saltos. Esta etapa me ayudó a entender con más claridad cómo un programa de alto nivel se transforma en instrucciones de bajo nivel.

20.2 Dificultades principales

Una de las dificultades principales fue resolver errores de gramática en PLY. Cuando una regla estaba mal conectada, el error podía aparecer lejos de la causa real, por lo que fue necesario revisar cuidadosamente las producciones.

Otra dificultad fue mantener consistente la tabla de símbolos. Al agregar arreglos, los valores guardados en `symbol_table` cambiaron de dos elementos a tres. Eso obligó a actualizar variables, parámetros y accesos para evitar errores.

También fue difícil generar correctamente estructuras de control. Cada `if`, `while`, `for`, `do/while` y `switch` requiere bloques distintos y saltos correctos. Un `branch` mal colocado puede dejar el IR inválido o cambiar el comportamiento del programa.

20.3 Decisiones de diseño importantes

Una decisión importante fue separar `analysis.py` y `arbol.py`. Esto hizo que el lexer y parser estuvieran en un archivo, mientras que el AST y el generador de código estuvieran en otro. La separación facilitó la lectura y depuración.

Otra decisión fue usar dos pasadas para funciones. Sin esta estructura, las llamadas recursivas y las llamadas a funciones definidas antes o después habrían sido más difíciles de manejar.

También decidí centralizar conversiones de tipos en funciones auxiliares. Esto evitó duplicar lógica y permitió que asignaciones, llamadas y retornos usaran reglas consistentes para aceptar o rechazar conversiones.

20.4 Relación entre teoría y práctica

En teoría, las etapas de un compilador se explican de forma separada: análisis léxico, análisis sintáctico, análisis semántico y generación de código. En la práctica, entendí que todas esas etapas están conectadas y que un error en una puede afectar a las demás.

La gramática de CLite sirvió como base para estructurar el lenguaje. Sin embargo, al implementar el compilador fue necesario adaptar algunas reglas, agregar extensiones y tomar decisiones prácticas para que los programas de prueba pudieran compilarse.

El uso de LLVM IR también hizo más concreta la teoría. Conceptos como variables, ciclos y condicionales se transforman en `alloca`, `load`, `store`, bloques y branches. Ver esa traducción me ayudó a entender mejor qué hace realmente un compilador.

20.5 Conclusión personal del proyecto

Al finalizar el proyecto, considero que logré construir una versión funcional de un mini compilador con varias características representativas. El compilador puede leer programas, construir un AST y generar LLVM IR para múltiples ejemplos.

También comprendí que un compilador completo requiere muchas validaciones y detalles. Incluso características aparentemente simples, como `printf`, arreglos o funciones void, implican decisiones de diseño y manejo cuidadoso de tipos.

El proyecto me dejó una base sólida para seguir aprendiendo sobre compiladores. Después de esta implementación, tengo una visión más clara de cómo se puede extender el lenguaje, mejorar los errores y acercar el compilador a una herramienta más completa.

XXI. Conclusión del Reporte

El desarrollo de este compilador permitió integrar las etapas principales de compilación en un proyecto funcional. A partir de código fuente similar a C, el compilador realiza análisis léxico, análisis sintáctico, construcción del AST, validaciones semánticas básicas y generación de LLVM IR.

El proyecto cubrió las construcciones solicitadas en la actividad: estructuras de control como `if/else`, `while`, `for`, `do/while` y `switch`; definición y llamado de funciones; conversión implícita de tipos; soporte para `printf`; y programas de prueba con llamadas recursivas. Además, incorporé extensiones como `void`, `char`, `string`, arreglos, bloques como sentencias y lectura de ejemplos desde archivos.

Aunque todavía existen limitaciones, el compilador cumple con el propósito de demostrar cómo se construye un lenguaje representativo hasta la generación de código intermedio. El resultado final no solo fue una implementación técnica, sino también una experiencia de aprendizaje sobre diseño de lenguajes, representación interna, memoria, control de flujo y generación de instrucciones LLVM.